LECTURE

Kalman Filter Basics and Applications

From the update equations to beta estimation and pairs trading

Concept · Theory · Equations · Examples · KF vs moving average · Regression beta · Pairs trading · VIX & KOSPI

How an optimal recursive estimator recovers the true state from noisy measurements —
and how traders use it for adaptive averages, time-varying betas and hedge ratios.

The Kalman filter fuses noisy sensors

What is the Kalman filter? — origin and typical applications

Kalman filter

- Named after Rudolf Kalman (1930–2016)
- Famous for its use in the Apollo program
- Typical uses: GPS, computer vision, signal processing
- Figure: IMU, odometer and GPS combined into one position estimate

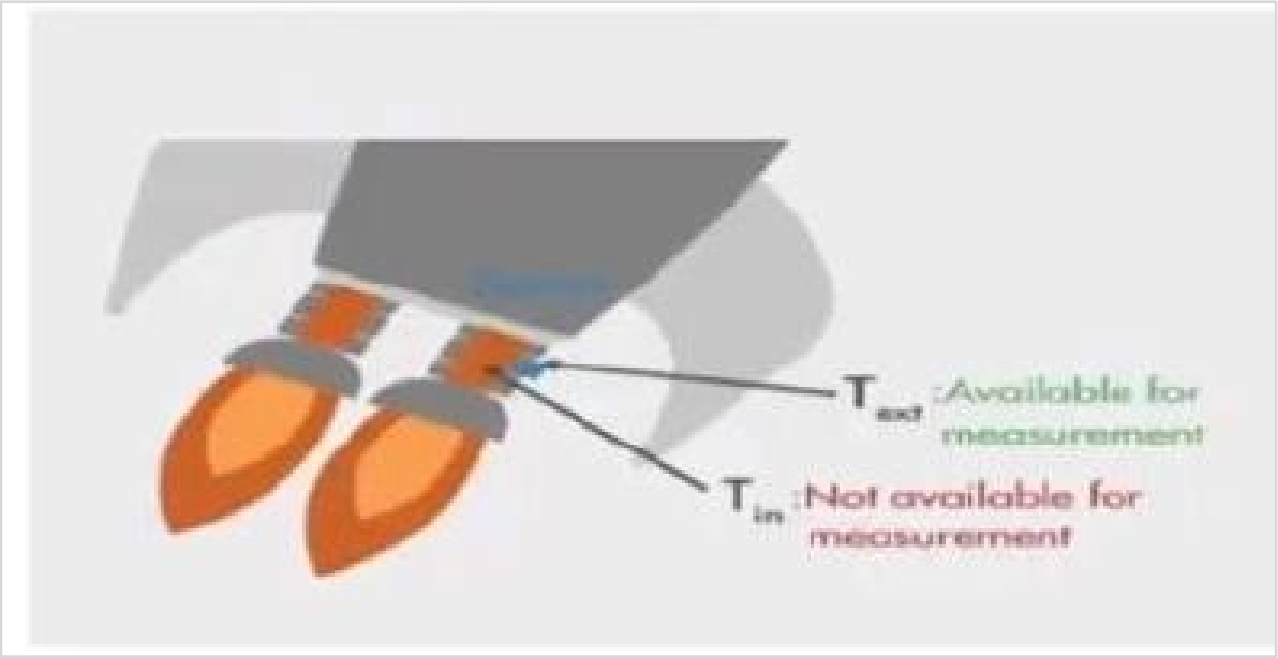


An optimal estimator, not machine learning

What is the Kalman filter? — key properties

Key properties

- Optimal estimation algorithm
- Not regarded as machine learning
- Estimates past, present and future states
- Works even when the exact nature of the system is unknown
- Useful when the variable is measured only indirectly
- ...or measured from several noisy sources

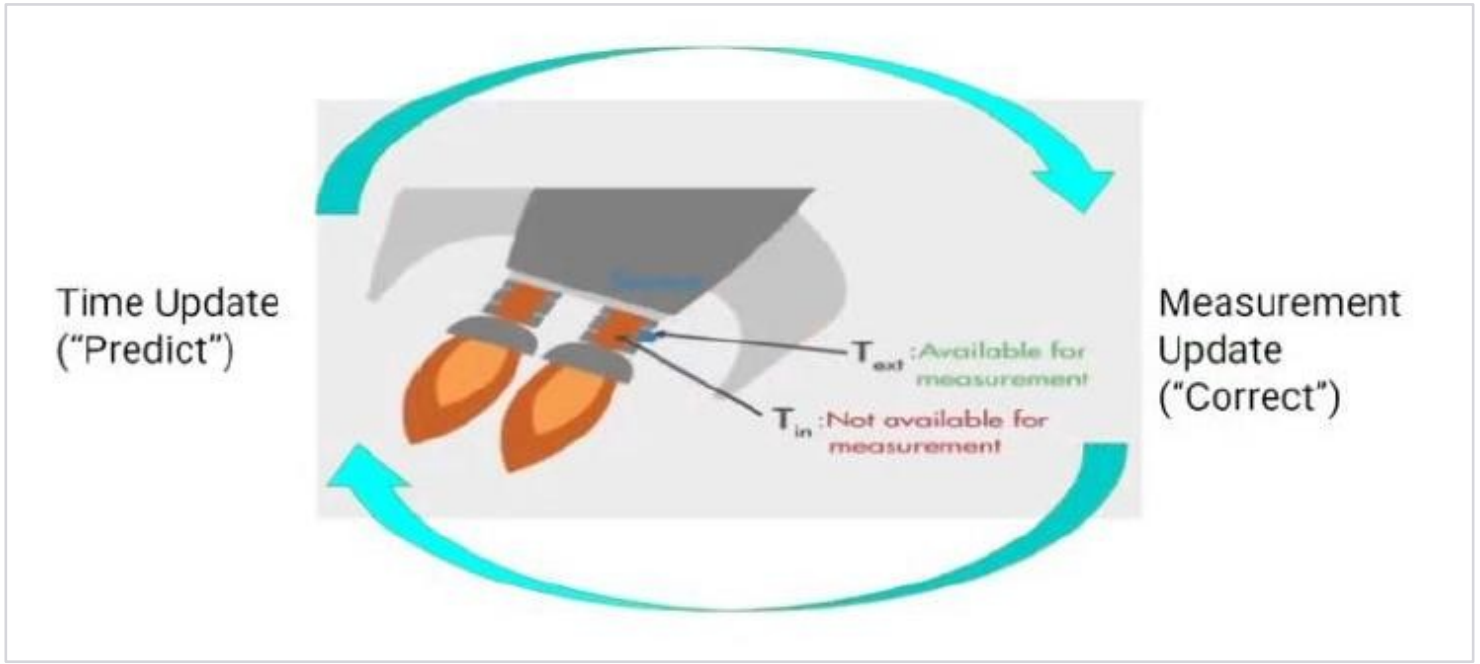


The filter recovers the true underlying state

What is the Kalman filter? — time update (predict) and measurement update (correct)

Idea

- Given a sequence of noisy measurements, the KF recovers the “true state” of the underlying object it tracks
- Loop: time update (“predict”) ↔ measurement update (“correct”)

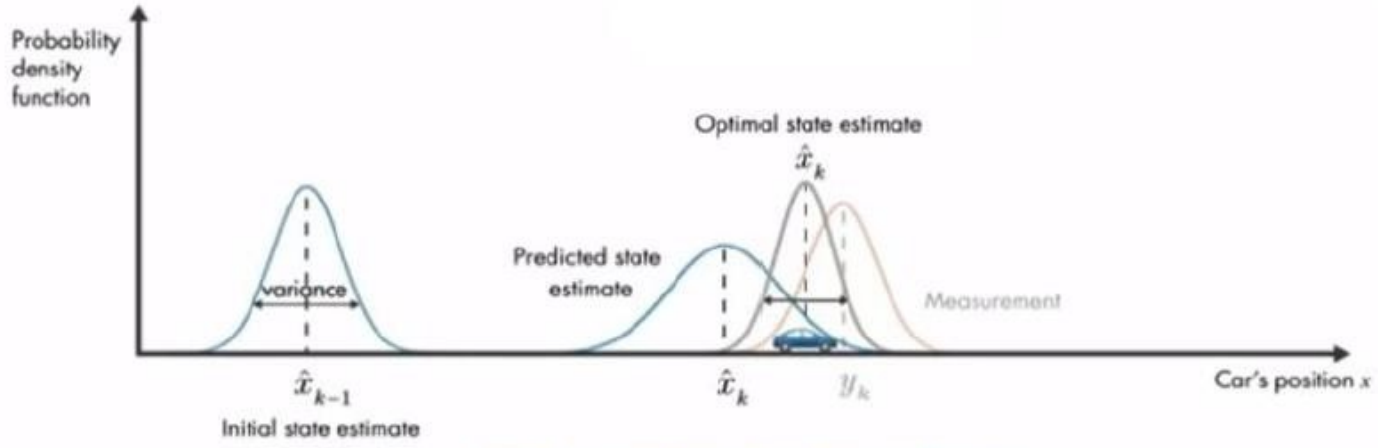


Prediction and measurement combine into one

Kalman filter theory — estimating a car's position with a GPS sensor

Goal

- Estimate the car's position with a GPS sensor
- Use state estimates observed over several time steps
- Find the best estimate of the car's actual position



Source: https://www.youtube.com/watch?v=VFXf1lIZ3p8

$$p(x_k \mid y_{1:k}) = \frac{p(y_k \mid x_k, y_{1:k-1})\, p(x_k \mid y_{1:k-1})}{p(y_k \mid y_{1:k-1})}$$

$$p(x_k \mid y_{1:k}) = \frac{p(y_k \mid x_k)\, p(x_k \mid y_{1:k-1})}{p(y_k \mid y_{1:k-1})}$$

Three inputs and two models drive the filter

Kalman filter — objective

Goal

- Predict the “state” of an object
- e.g., a car's position or an asset's price

Three known variables

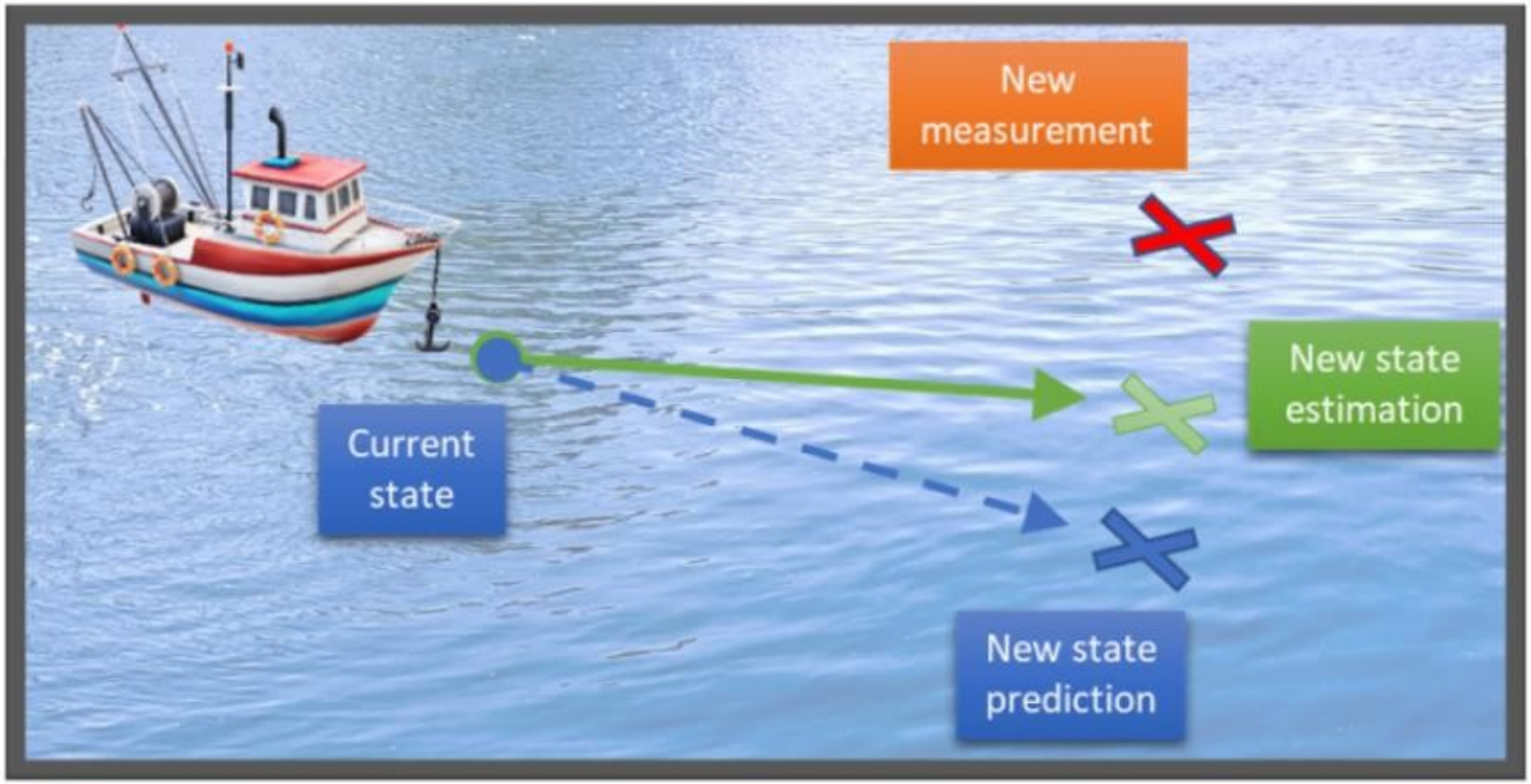
- Previous state estimate
- Model-based prediction of the true state
- Measurement of the true state

Two models

- State equation (state transition equation)
- Measurement equation

The new estimate lies between model and data

Kalman filter — current state, predicted new state, new measurement, new estimate



Each cycle predicts first, then corrects

The overall Kalman filter process

1. Prediction step

Compute the estimate and its uncertainty from previous data.

2. Correction (update) step

Compare the estimate with the actual measurement and update the estimate.

Predict → measure → correct → repeat

01

PART 1

Kalman Filter Explained

From a running average to the gain and the error covariance

An update is the old estimate plus a gain

Understanding the update — a running average rewritten as an innovation step

Recursive average

$\hat{u}_{99}$: Average of $b_1, b_2, \ldots, b_{99}$

$b_{100} \rightarrow \hat{u}_{100}$

$$\hat{u}_{100} = \frac{99}{100}\hat{u}_{99} + \frac{1}{100}b_{100}$$

$$= \hat{u}_{99} + \frac{1}{100}(b_{100} - \hat{u}_{99})$$

Innovation and gain factor

$b_{100} - \hat{u}_{99}$: innovation

$\frac{1}{100}$: gain factor

The new estimate moves from the old one toward the new data by a fraction set by the gain

The KF is a low-pass filter with a dynamic α

Kalman filter explained — comparison with the average

Average

$$
\begin{aligned}
&Average: \ \bar{X}_K = \frac{X_1 + X_2 + \cdots + X_K}{K} \ \Rightarrow \ \bar{X}_{K-1} = \frac{X_1 + X_2 + \cdots + X_{K-1}}{K-1} \\
&\Rightarrow \ K\,\bar{X}_K = X_1 + X_2 + \cdots + X_{K-1} + X_K = (K-1)\,\bar{X}_{K-1} + X_K \\
&\Rightarrow \ \bar{X}_K = \left(\frac{K-1}{K}\right)\bar{X}_{K-1} + \left(\frac{1}{K}\right)X_K
\end{aligned}
$$

1st order low pass filter

$$
\bar{X}_K = \alpha \bar{X}_{K-1} + (1-\alpha)\,X_K
$$

Kalman Filter (α adjusted dynamically)

$$
\begin{aligned}
\hat{X}_t &= \hat{X}_t^- + K_t(y_t - H\,\hat{X}_t^-) \\
&= (1-K_t)\,\hat{X}_t^- + K_t\,y_t \quad (\text{assuming } H=1) \\
&= \alpha \hat{X}_t^- + (1-\alpha)\,y_t
\end{aligned}
$$

The algorithm runs predict, gain, estimate, cov

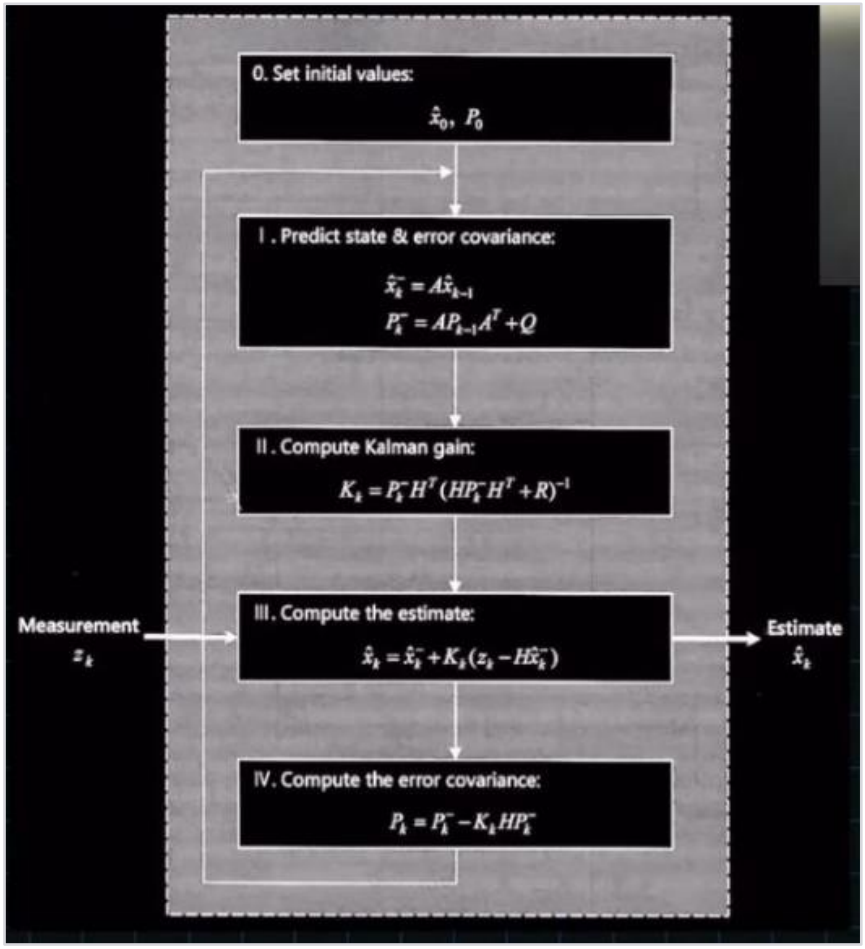
Kalman filter explained — the Kalman filter algorithm

I. Prediction step

$$X_t,\ \hat{X}_t,\ \hat{X}_t^-$$

$$X_t \sim \mathrm{N}(\hat{X}_t,\ P_t)$$

II–IV: Estimation step



With H = 1, the update is a weighted average

Kalman filter explained — II–IV: Estimation step and the Kalman gain

II–IV: Estimation step

$$\hat{X}_t = \hat{X}_t^- + K_t(y_t - H\,\hat{X}_t^-)$$
$$\hat{X}_t = \hat{X}_t^- + K_t y_t - K_t H\,\hat{X}_t^- = (I - K_t H)\,\hat{X}_t^- + K_t y_t$$
$$\text{If } n = 1,\ H = 1:$$
$$\hat{X}_t = (I - K_t)\,\hat{X}_t^- + K_t y_t$$
$$\text{Using } \alpha \text{ instead of } (I - K_t):$$
$$\hat{X}_t = \alpha\,\hat{X}_t^- + (1-\alpha) y_t \ : \ \text{Kalman Filter}$$
$$\hat{X}_t = \alpha \hat{X}_{t-1} + (1-\alpha) X_t \ : \ \text{Low Pass Filter}$$

How to get the gain?

Gain, updated at each time t

$$K_t = P_t^- H^T (H P_t^- H^T + R)^{-1}$$

Error cov updated to

$$P_t = P_t^- - K_t H P_t^-$$

A and Q act only in the prediction step

Kalman filter explained — I. Prediction step; H and R appear only in the estimation step

$$\text{True state } X_t \sim \mathrm{N}(\hat{X}_t,\ P_t)$$

I. Prediction step

$\hat{X}_t$ at time $t \Rightarrow \hat{X}_{t+1}$ at time $t+1$

$$\hat{X}_{t+1}^{-} = A\hat{X}_t$$

$$P_{t+1}^{-} = AP_tA^T + Q$$

$\hat{X}_t$: estimates of state

$\hat{X}_t^{-}$: prediction of state

P_t : estimates of error cov

P_t^{-} : prediction of error cov

Role	Symbols
External input	z_t (measurement)
Final output	$\hat{X}_t$ (estimate)
System model	$A,\ H,\ Q,\ R$
Internal computation	$\hat{X}_t^{-},\ P_t^{-},\ P_t,\ K_t$

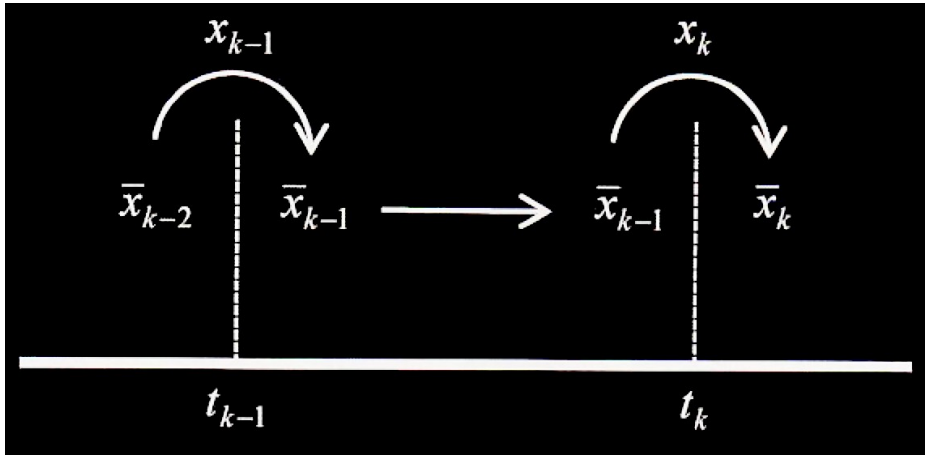
R: measurement error · Q: state transition noise

The KF adds a model-based prediction step

Kalman filter explained — how do prediction and estimates fit together?

A. Low Pass Filter

$$\hat{X}_t = \alpha \hat{X}_{t-1} + (1-\alpha) X_t$$

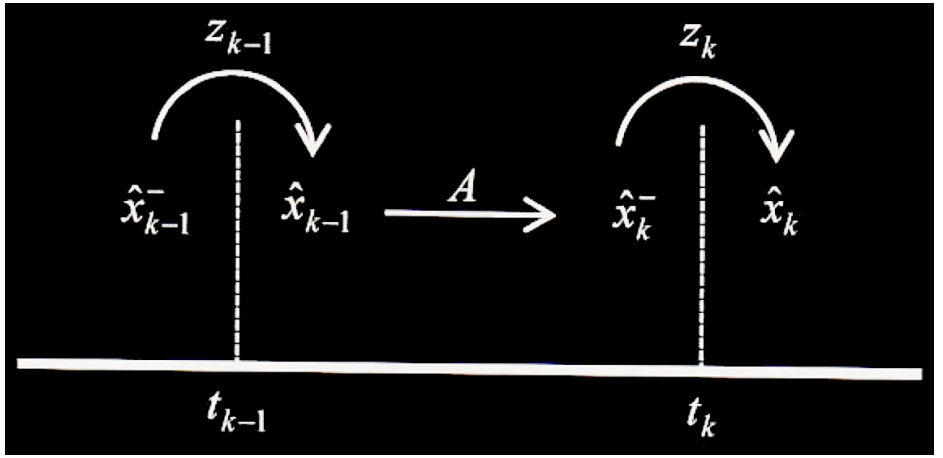


B. Kalman Filter

$$\hat{X}_t = \hat{X}_t^- + K_t(y_t - H\,\hat{X}_t^-) \quad \Big| \quad \hat{X}_t^- = A\hat{X}_{t-1}$$

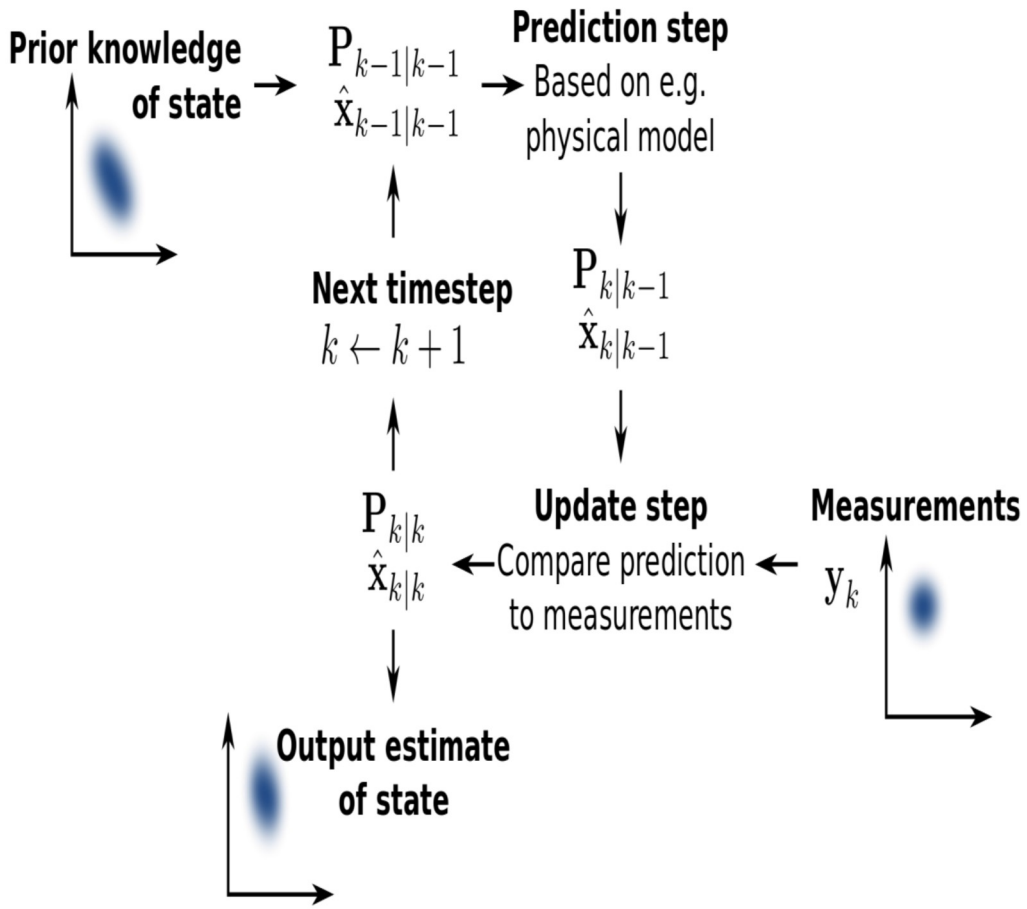
$$\hat{X}_t = \hat{X}_t^- + K_t(y_t - HA\hat{X}_{t-1})$$

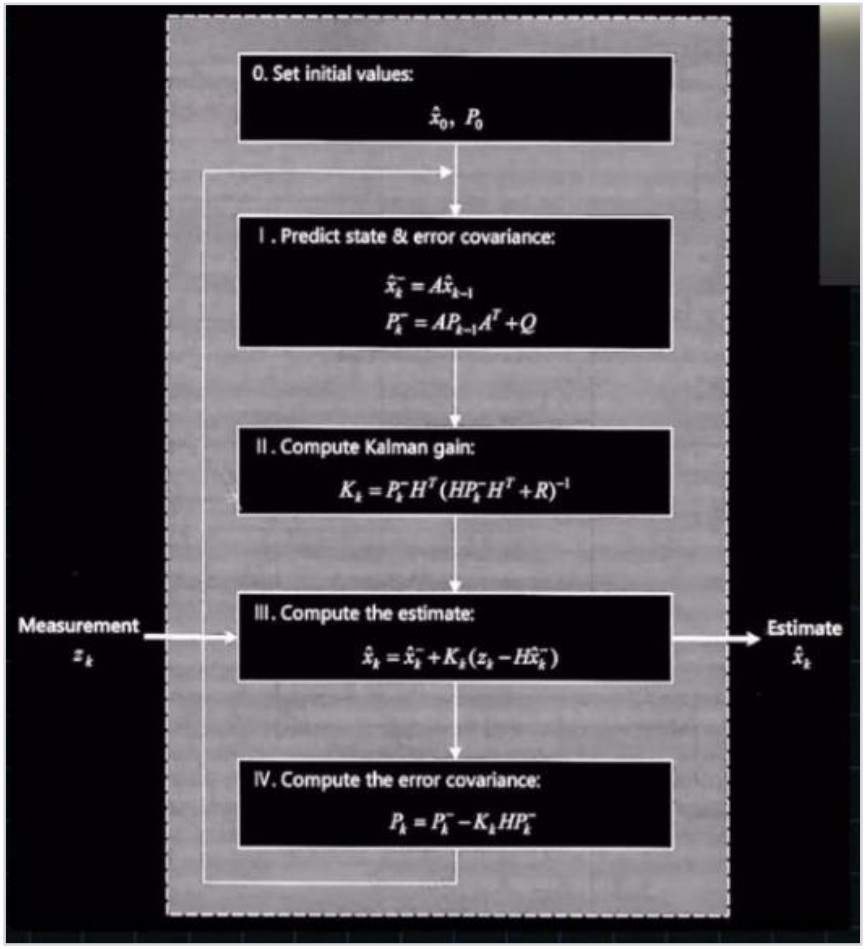
$$y \;\rightarrow\; z$$



Prior, predict, update, output — then repeat

Kalman filter process — summary (left: distribution view, right: algorithm)





A constant state needs only R and a first guess

Kalman filter example — system model parameters: A, H, Q, R

Linear state model

$$\text{State eq: } X_{t+1} = AX_t + w_t \quad : \text{ Linear and process noise}$$
$$\text{Meas eq: } y_t = HX_t + v_t \quad : \text{ Linear and measurement noise}$$

Example

$$X_{t+1} = X_t$$
$$y_t = X_t + v_t$$

$$\Rightarrow X_{t+1} = AX_t,\ A = 1,\ H = 1,\ Q = 0,\ R = 4$$
$$\Rightarrow \text{Need initial guess: } \hat{X}_0^- = 14,\ P_0^- = 6$$

EXAMPLE 2

Position and velocity form a two-state model

Kalman filter example — Example 2 (n = 2): only the position is measured

$$X_t = \begin{bmatrix} position \\ velocity \end{bmatrix}$$
$$X_{t+1} = AX_t + w_t$$
$$y_t = HX_t + v_t$$

$$A = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \ (2 \times 2), \qquad H = \begin{bmatrix} 1 & 0 \end{bmatrix} \ (1 \times 2)$$

$$X_{t+1} = \begin{bmatrix} pos \\ vel \end{bmatrix}_{t+1} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} pos \\ vel \end{bmatrix}_t = \begin{bmatrix} pos_t + \Delta t * vel_t \\ vel_t \end{bmatrix}_t$$

$$y_t = HX_t + v_t = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} pos \\ vel \end{bmatrix}_t + v_t \ \Rightarrow \ y_t = pos_t + v_t$$

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} \qquad R = 10$$

Initial values

$$X_0^- = \begin{bmatrix} 0 \\ 20 \end{bmatrix}, \quad P_0^- = \begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix}$$

02

PART 2

Kalman Filter Summary

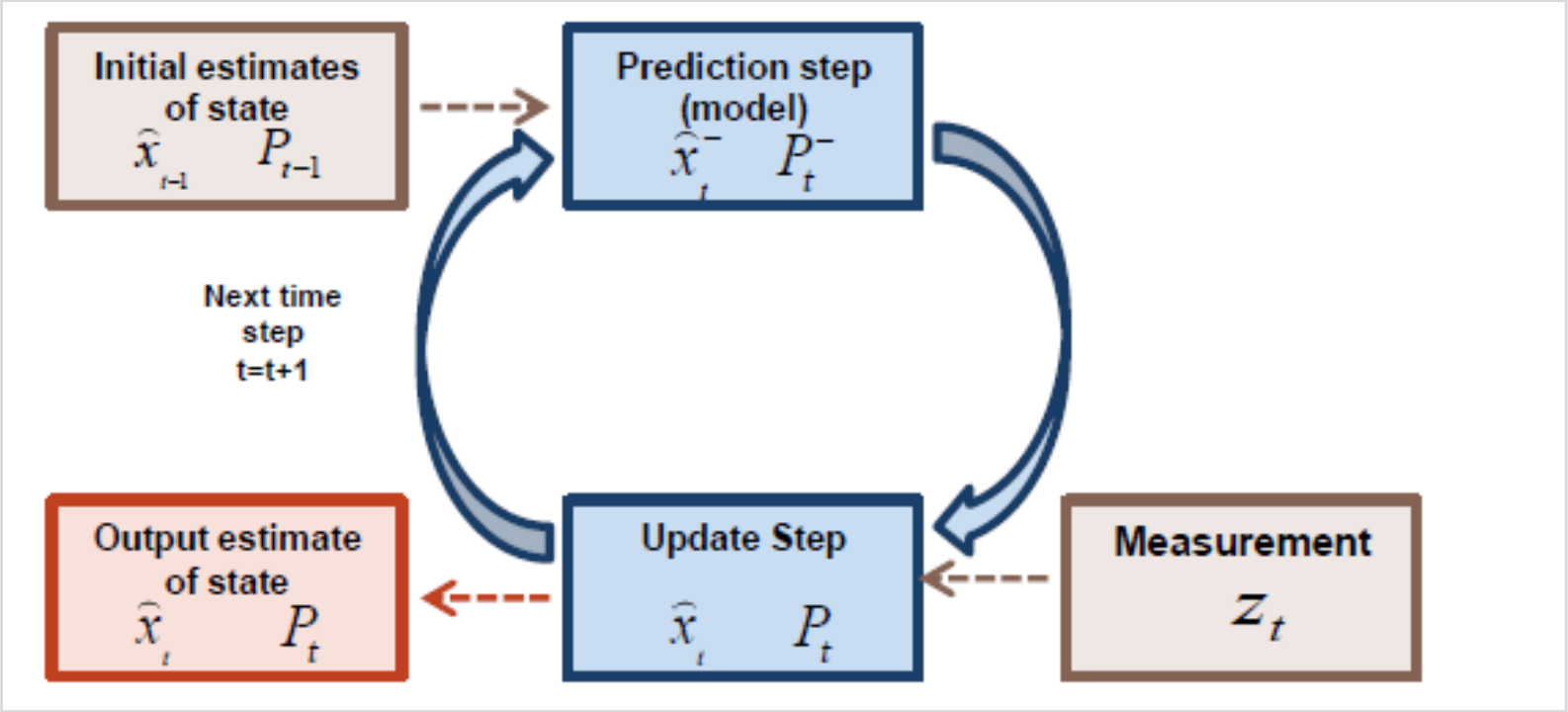
Process, state update, equations and filter types

Estimates flow in a predict–update loop

Overall process — the Kalman filter loop

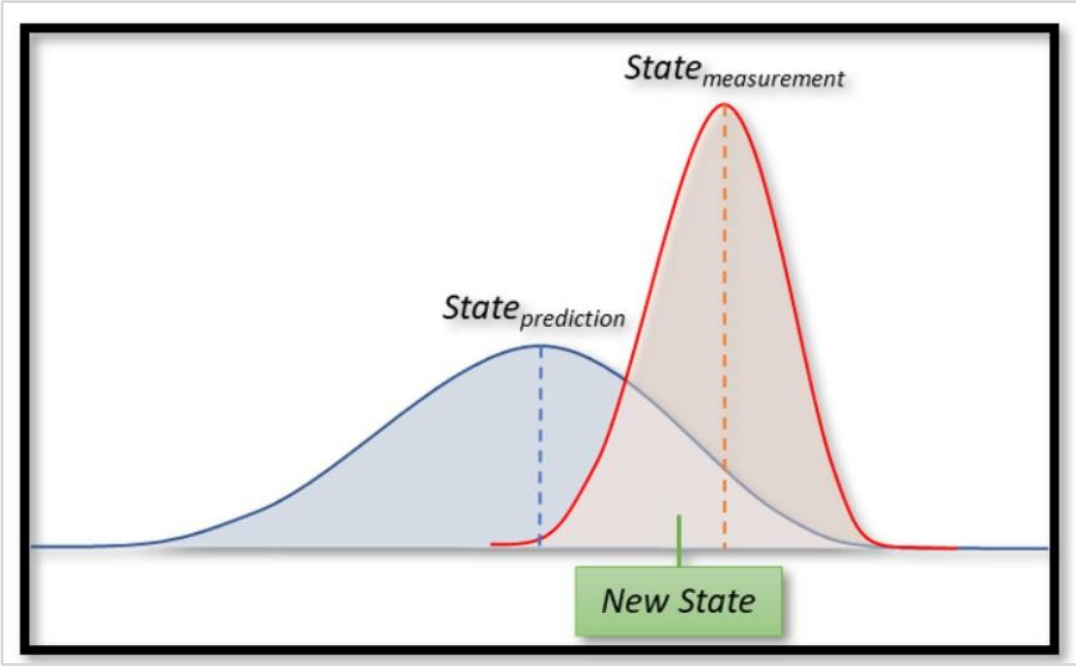
Reading the loop

- Start from initial estimates
- Prediction step uses the model
- Update step uses the measurement
- Output the estimate, then move to the next time step



The gain weights prediction against measurement

Kalman filter — state update as a weighted average (Kalman gain)



Kalman gain (between 0 and 1)

$$K_t = \frac{Uncertainty_{prediction}}{Uncertainty_{prediction} + Uncertainty_{measurement}} = [0, 1]$$

State update

$State = K_t * State_{measurement} + (1 - K_t) * State_{prediction}$, or:

$State = State_{prediction} + K_t(State_{measurement} - State_{prediction})$

A state and a measurement equation define it

Kalman equations — state equation, measurement equation and coefficients

State equation

$$x_t = Ax_{t-1} + Bu_t + w_{t-1}$$

where:

$x_t \rightarrow$ state we are trying to predict $(x \in \Re^n)$

$x_{t-1} \rightarrow$ previous state

$A \rightarrow n \times n$ matrix

$u_t \rightarrow$ (optional) command input $(u \in \Re^l)$

$B \rightarrow n \times l$ matrix

$w_{t-1} \rightarrow$ process noise

Measurement equation

$$z_t = Hx_t + v_t,$$

where:

$z_t \rightarrow$ the current measurement $z \in \Re^m$

$H \rightarrow m \times n$ matrix

$v_t \rightarrow$ measurement noise

Coefficients

$H:$ observation matrix (often $H = I$)

$$p(w) \sim N(0, Q)$$
$$p(v) \sim N(0, R)$$

The time update predicts state and covariance

Kalman filter theory — a repeated cycle of time update (prediction) and measurement update

1. Time update — state prediction

$$\hat{x}_t = Ax_{t-1} + Bu_t$$

Uncertainty (covariance) prediction

$$\hat{P}_t = A * P_{t-1}A^T + Q$$

The measurement update corrects with the gain

Kalman filter theory — 2. Measurement update: gain, state update, covariance update

Kalman gain

$$K_t = \hat{P}_t H^T (H \hat{P}_t H^T + R)^{-1}$$

State update

$$x_t = \hat{x}_t + K_t (z_t - H \hat{x}_t)$$

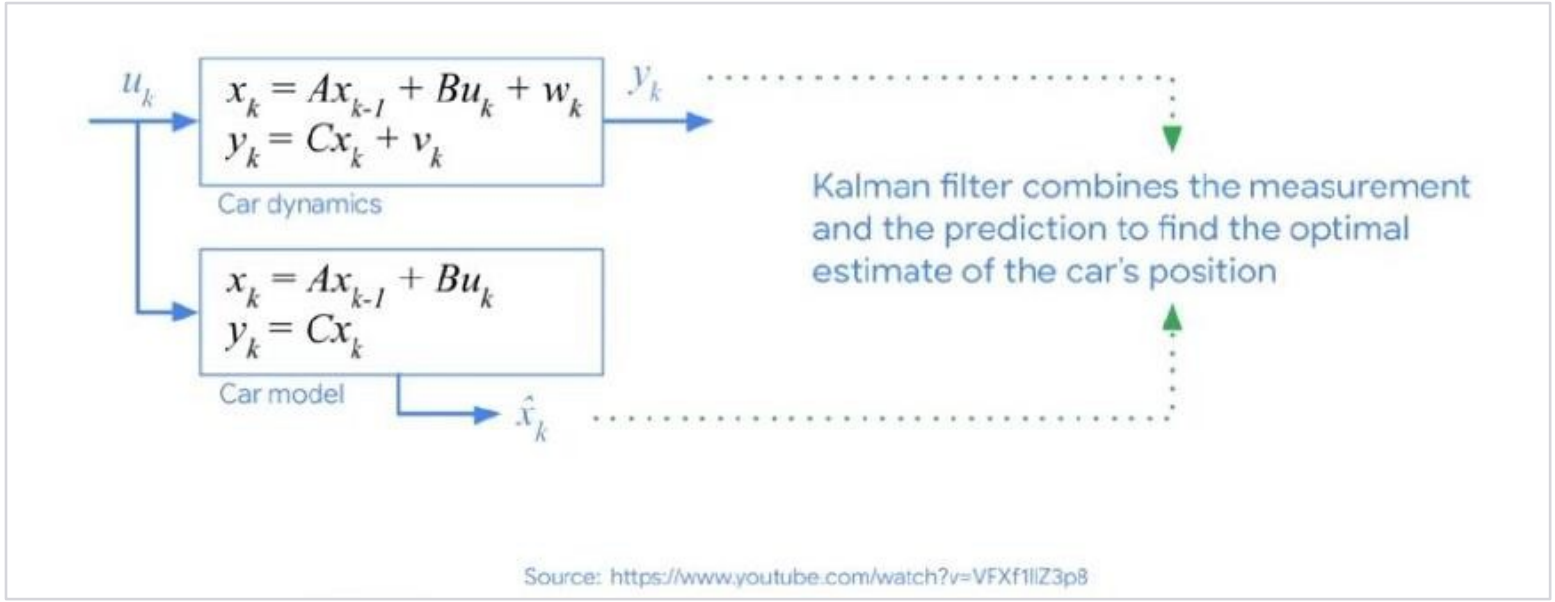
$$P_t = (I - K_t H) \hat{P}_t$$

The filter merges car dynamics and car model

Kalman filter theory — combining measurement and prediction to find the optimal car position

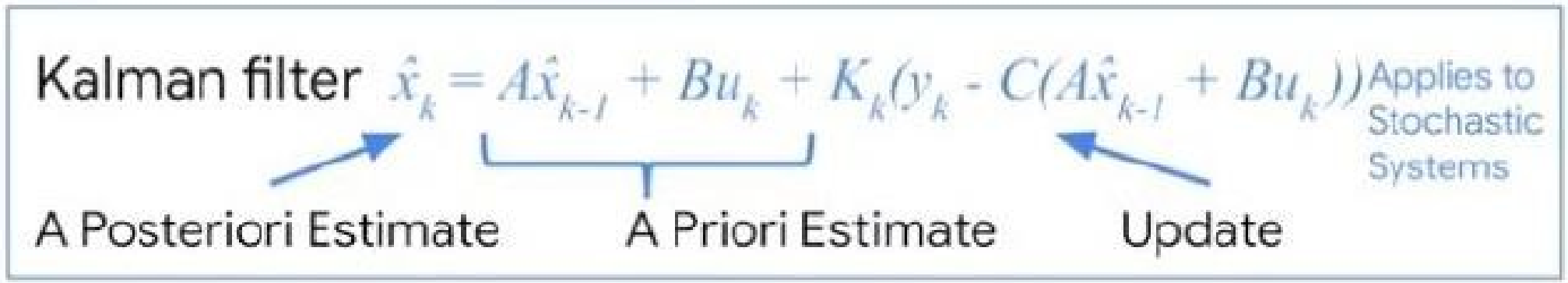
Two paths

- Car dynamics → measurement
- Car model → prediction
- The KF combines both into the optimal position estimate



Measure and predict repeat until convergence

Kalman filter theory — the measurement–prediction process repeats until the optimal state is found



A posteriori estimate

The result: the updated state estimate

A priori estimate

The model prediction before the measurement

Update

Gain-weighted correction by the measurement surprise

Source: https://www.youtube.com/watch?v=VFXf1llZ3p8

STRENGTHS

Recursive updates make the KF fast and robust

Overall process — advantages of the Kalman filter process

Missing data

Handles missing data well

Irregular series

Handles irregular time-series observations, e.g. a mix of monthly and weekly data

Fast reaction

Responds quickly to change

Recursive

No need to recompute from scratch — just update the model with new data

One cheap update per new observation

Filters differ in model, distribution and cost

Types of Kalman filters — KF, EKF, UKF, particle filter

State Estimator	Model	Assumed distribution	Computational cost
Kalman Filter (KF)	Linear	Gaussian	Low
Extended Kalman Filter (EKF)	Locally linear	Gaussian	Low (if the Jacobians need to be computed analytically) Medium (if the Jacobians can be computed numerically)
Unscented Kalman Filter (UKF)	Nonlinear	Gaussian	Medium
Particle Filter (PF)	Nonlinear	Non-Gaussian	High

Source: https://www.youtube.com/watch?v=Vefia3JMeHE

03

PART 3

Application: Replacing the Moving Average

Kalman estimate of the average vs. 50-day and 100-day moving averages

A Kalman average needs no window length

Kalman filter vs. moving average — AAPL, 50-day and 100-day moving averages

The Kalman filter...

- Updates the estimate at every time step
- Puts more weight on recent observations
- Needs no window length, unlike a moving average

In the example

- Kalman vs. 50-day and 100-day moving averages
- The estimate evolves like a random walk with a small error term
- The model estimates the mean of the next-value distribution
- The initial guess is mean = 0

pykalman turns prices into a Kalman average

Kalman filter vs. moving average — Claude Code prompt

[Input] Prompt · Claude Code (Code tab)

```
In the kalman-lab folder, do the following.
1. From Yahoo Finance, download AAPL and SPY adjusted
   closes from 2017-11-01 to 2019-11-01. Set x = AAPL.
2. Build a pykalman KalmanFilter:
   transition_matrices=[1], observation_matrices=[1],
   initial_state_mean=0, initial_state_covariance=1,
   observation_covariance=1, transition_covariance=0.01
3. Get the state means with filter, and compute the
   50-day and 100-day moving averages of x.
4. Plot the Kalman estimate, x and both moving averages
   on one chart.
```

[Check] What to look for

· Does it start at 0 and jump to the price level within a few steps?

· Does it then follow AAPL more closely than both moving averages?

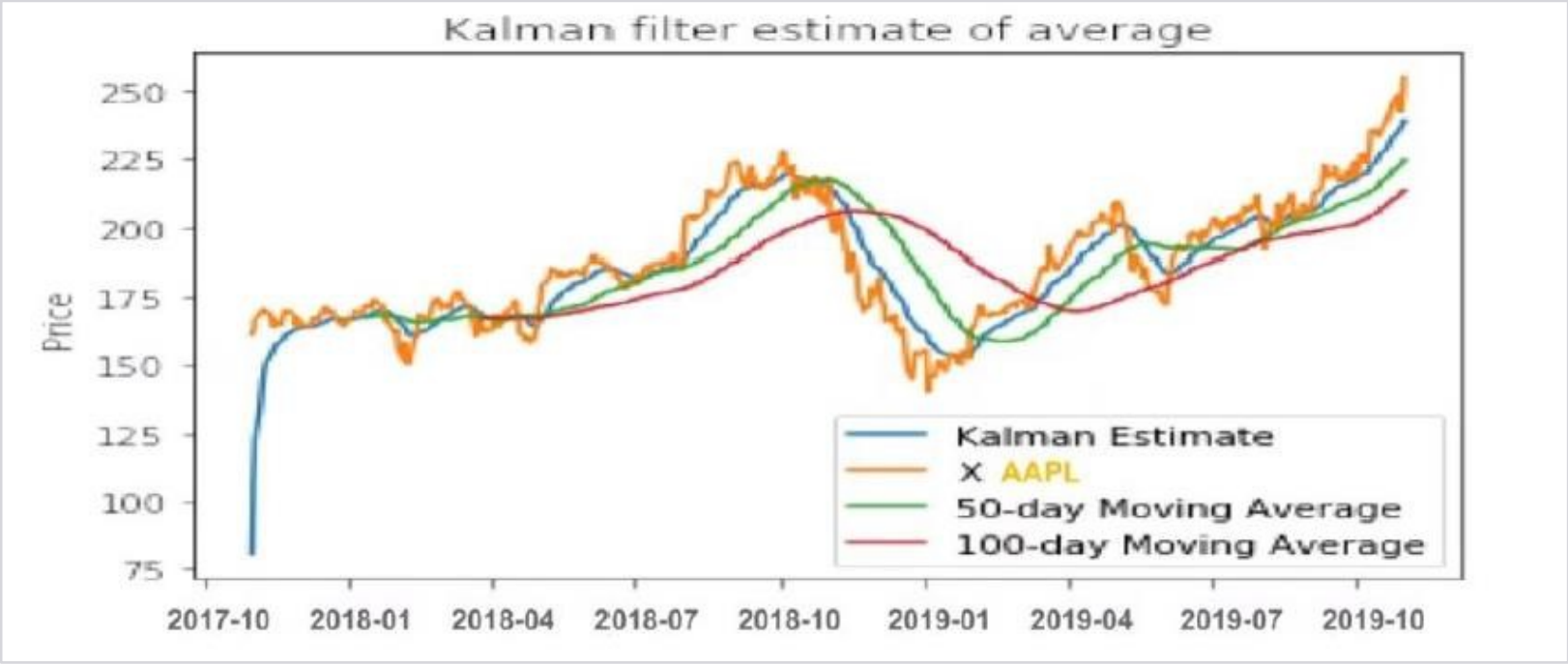
· Does the 100-day MA react slowest?

The KF estimate tracks prices with less lag

Kalman estimate vs. 50-day and 100-day moving averages (AAPL, 2017–2019)

What to see

- Starts at 0, jumps to the price level within a few steps
- Then follows AAPL more closely than both moving averages

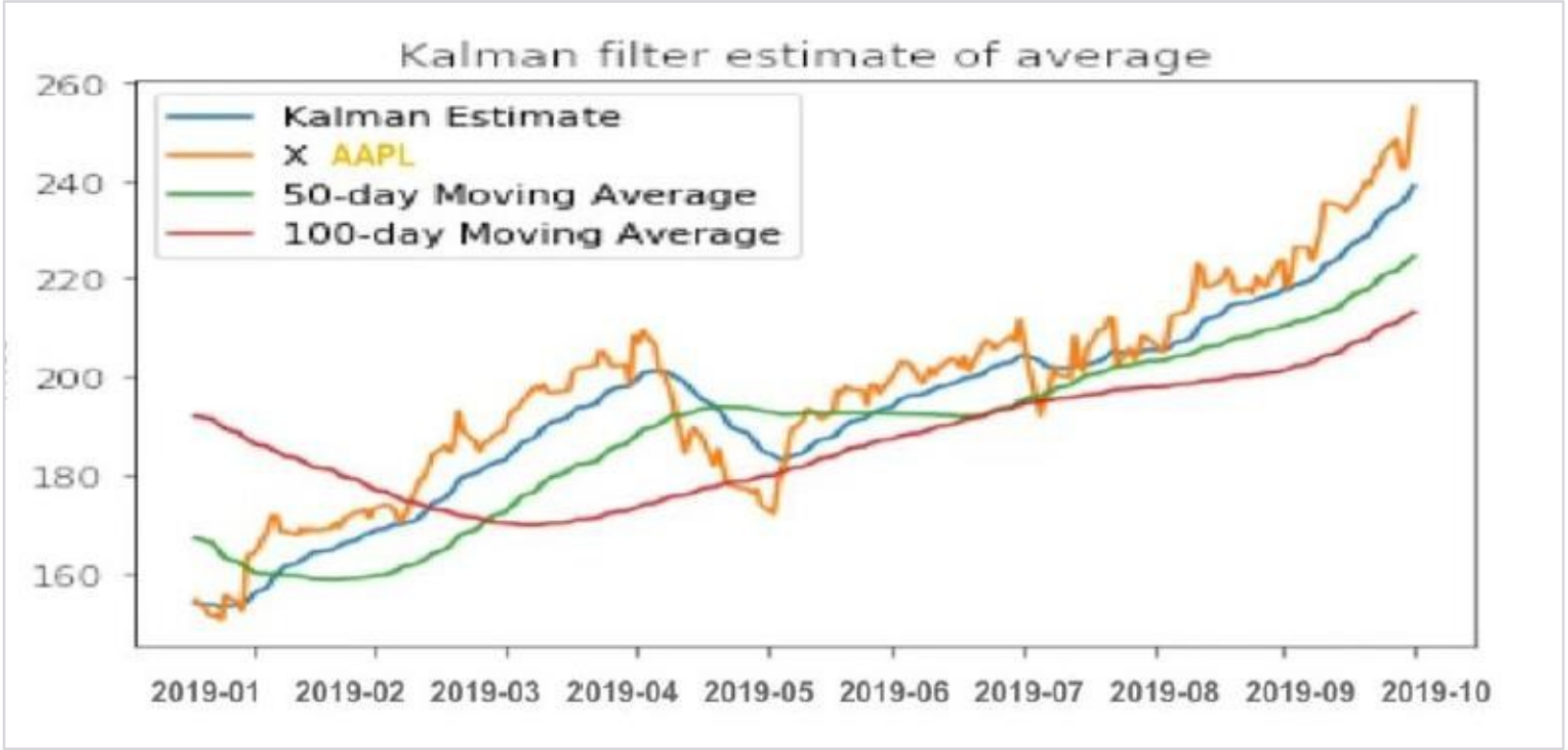


Moving averages lag at turning points

Kalman estimate vs. 50-day and 100-day moving averages (AAPL, 2019)

What to see

- The 100-day MA reacts slowest
- The Kalman estimate sits between price and the 50-day MA



04

PART 4

Example: Estimating Regression Beta

Time-varying α and β of AAPL against SPY

Treat the regression coefficients as the state

Estimating regression coefficients with the Kalman filter — inputs

Kalman filter inputs

- The initial system state is the line the observations follow
- The initial guess for α and β is (0, 0)
- The new system state is given by the equation on the right

New system state

$$(\beta, \alpha) \cdot (x_i, 1) \Rightarrow \beta\, x_i + \alpha$$

Intercept and slope follow a random walk

Estimating regression coefficients — observation and state equations

Observation equation

$$y_t = b_t x_t + a_t + v_t$$

y_t : observation (actual value)

x_t : independent variable (input)

v_t : observation noise, $\mathcal{N}(0, R)$

State equation

$$\begin{bmatrix} a_t \\ b_t \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} a_{t-1} \\ b_{t-1} \end{bmatrix} + w_t$$

a_t : intercept, hidden state

b_t : slope, hidden state

w_t : state noise, $\mathcal{N}(0, Q)$

A 2-D state holds α and β; prices are observed

Estimating regression coefficients — prompt to estimate and plot α and β

[Input] Prompt · Claude Code (Code tab)

```
In the same folder, continue as follows.
1. Set x to the SPY price and y to the AAPL price.
2. Set delta=1e-3 and the transition covariance to
   delta/(1-delta) × the 2×2 identity matrix.
3. Build the observation matrix as [x, 1] at each time.
4. Build a KalmanFilter with n_dim_obs=1, n_dim_state=2.
   State order (β, α), initial_state_mean=[0, 0],
   initial_state_covariance all ones (2×2),
   transition_matrices the identity,
   observation_covariance=2.
5. Run filter(y) for running estimates and plot slope
   and intercept in two stacked panels.
```

[Check] What to look for

· Is the first state component the slope (β)?

· Does beta change a lot over time while alpha drifts only slightly?

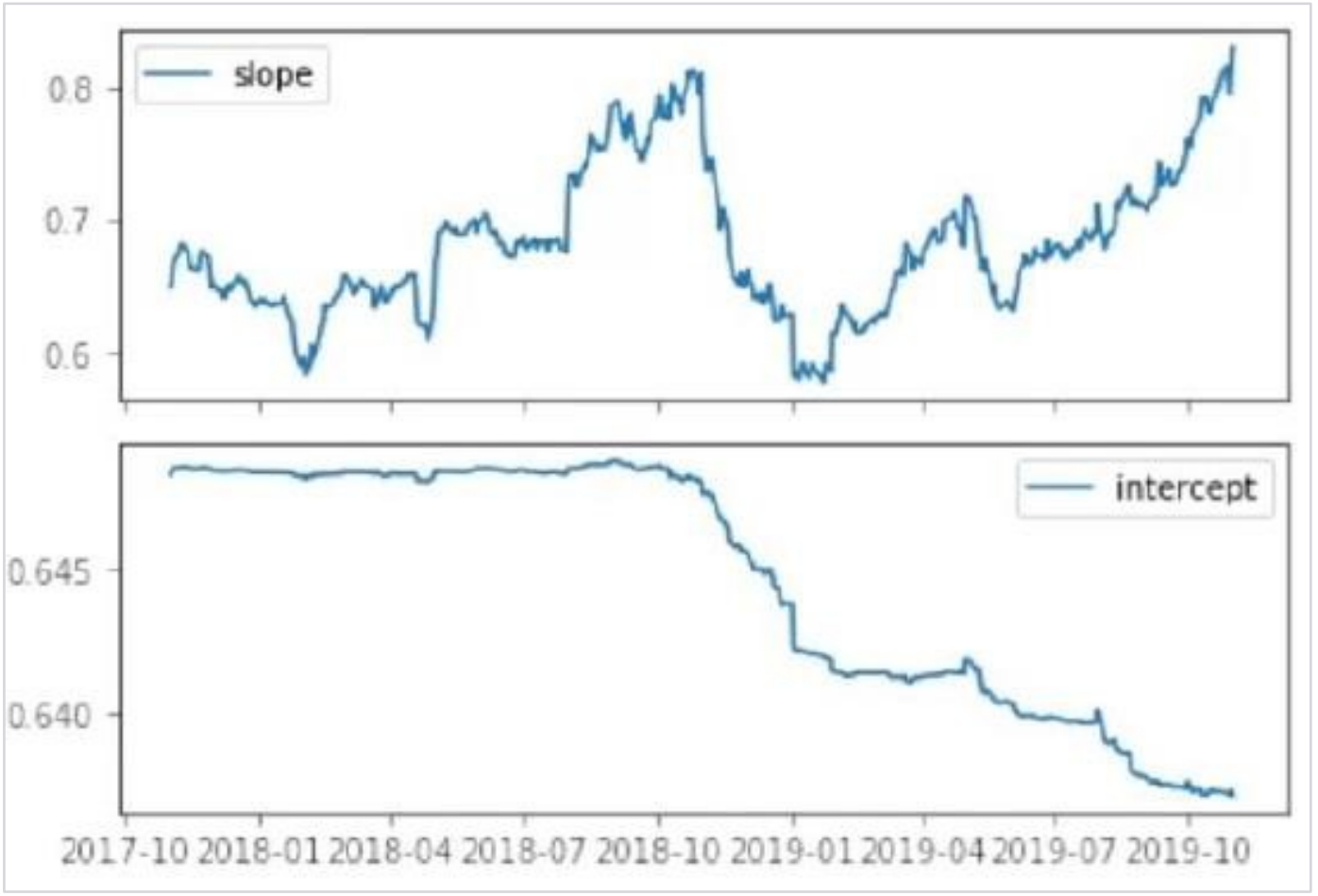
· Do the estimated lines fit recent data better than old data?

Beta varies strongly over time; alpha barely moves

Regression coefficients of AAPL vs. SPY

Implication

- Beta changes a lot over time; alpha drifts only slightly
- If beta hedges a trading strategy, using the current, accurate beta is essential

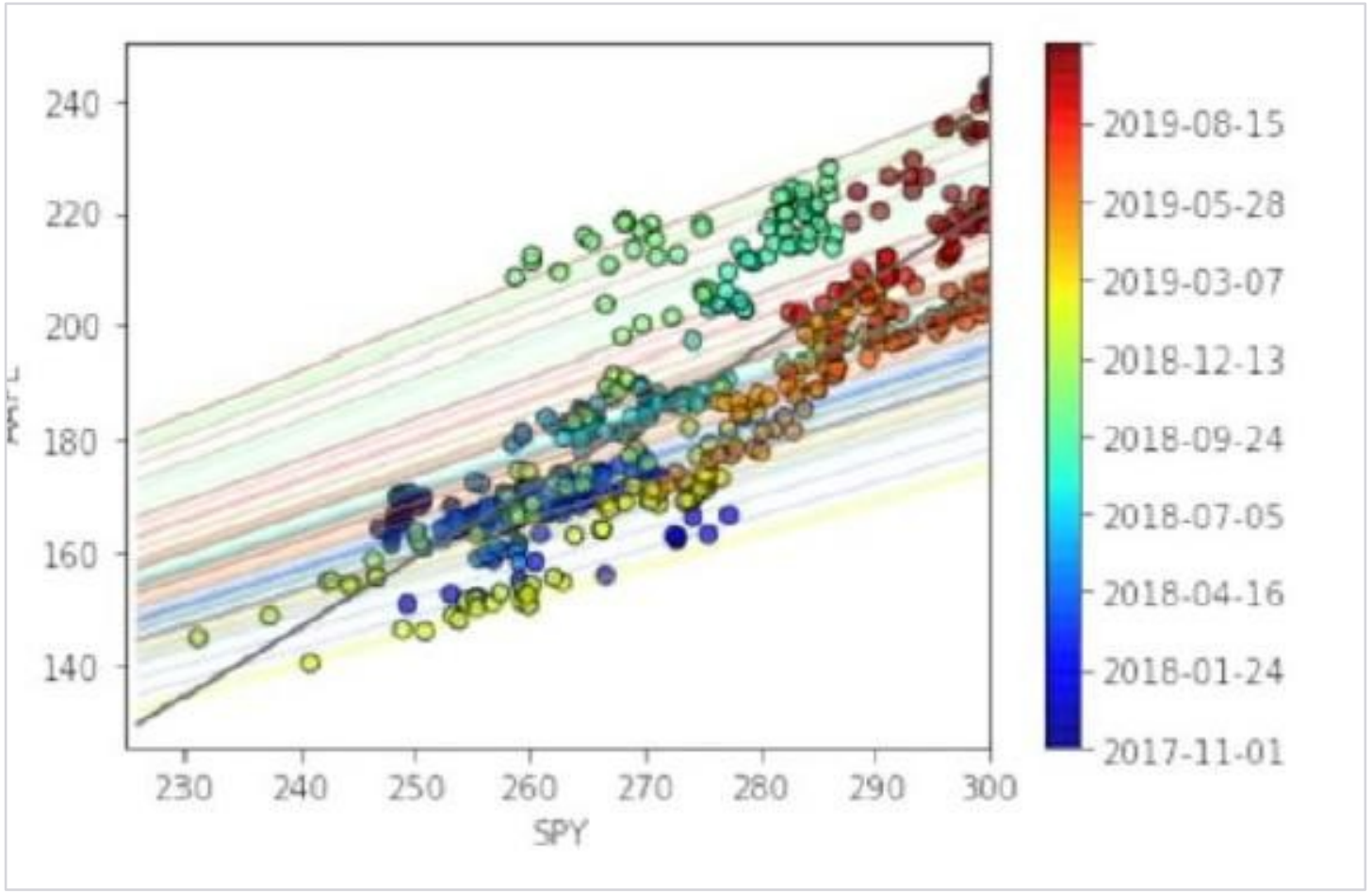


Recent data dominate each estimated line

Regression coefficients of AAPL vs. SPY — lines colored by date

Reading the chart

- Each estimate uses all past observations
- ...but fits recent data better than old data
- So the filter adapts to structural change



The same filter runs on returns

Prompt for the beta chart using AAPL and SPY returns

[Input] Prompt · Claude Code (Code tab)

```
In the same folder, continue as follows.
1. Turn x and y into returns with pct_change and drop
   the first row.
2. Set delta=1e-2 and the transition covariance to
   delta/(1-delta) × the 2×2 identity matrix.
3. Build the observation matrix as [x return, 1].
4. Use the same 2-D KalmanFilter with
   observation_covariance=0.01.
5. Run filter on the y returns and plot slope and
   intercept in two panels.
6. On a returns scatter, draw the estimated lines colored
   by date and overlay the OLS line in black.
```

[Check] What to look for

· Are returns less dispersed?

· Does the Kalman slope start near 0?

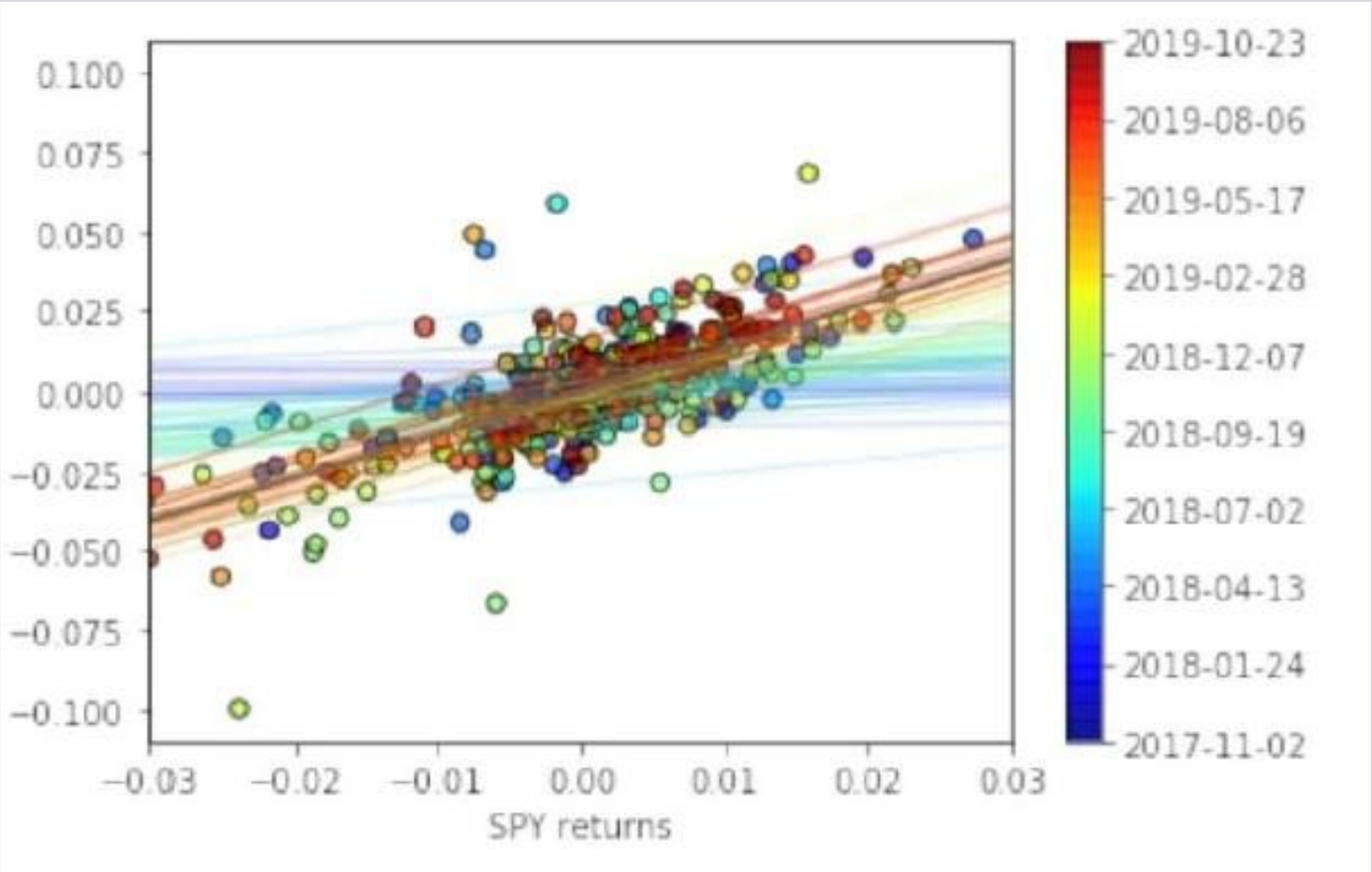
· Does it rise over time to equal the OLS coefficient (black line)?

On returns the slope converges to OLS

Regression coefficients of AAPL vs. SPY returns

Reading the chart

- Returns are less dispersed
- The Kalman slope starts near 0
- It rises over time to equal the OLS coefficient (black line)



05

PART 5

Example: Pairs Trading

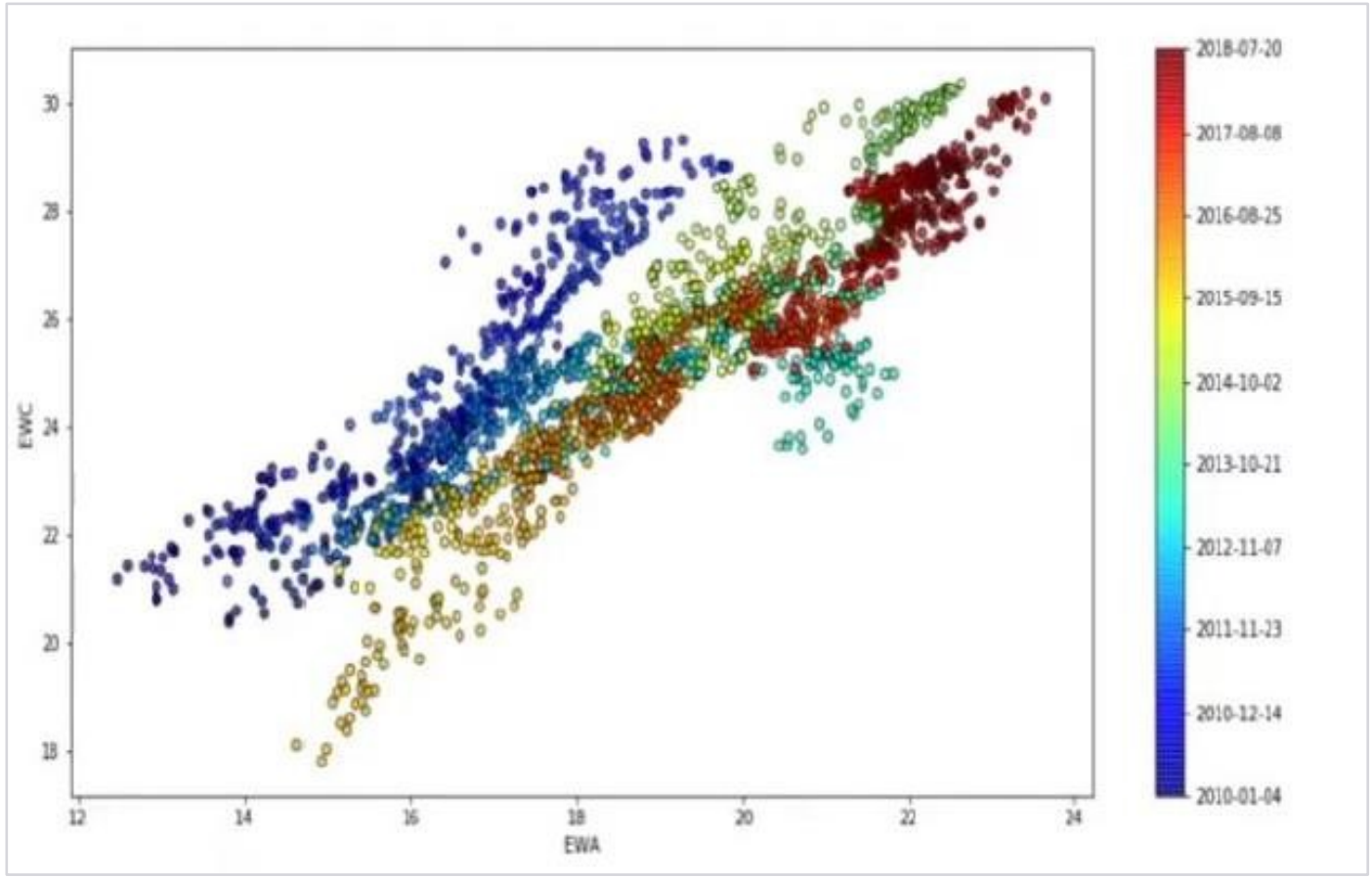
EWA vs. EWC — Ernest Chan's dynamic hedge ratio

Two highly correlated ETFs: can one predict the other?

Kalman filter example — forecasting stock correlation

Question

- Assume EWA and EWC are highly correlated
- If we know one, can we predict the other's price?
- Color = date



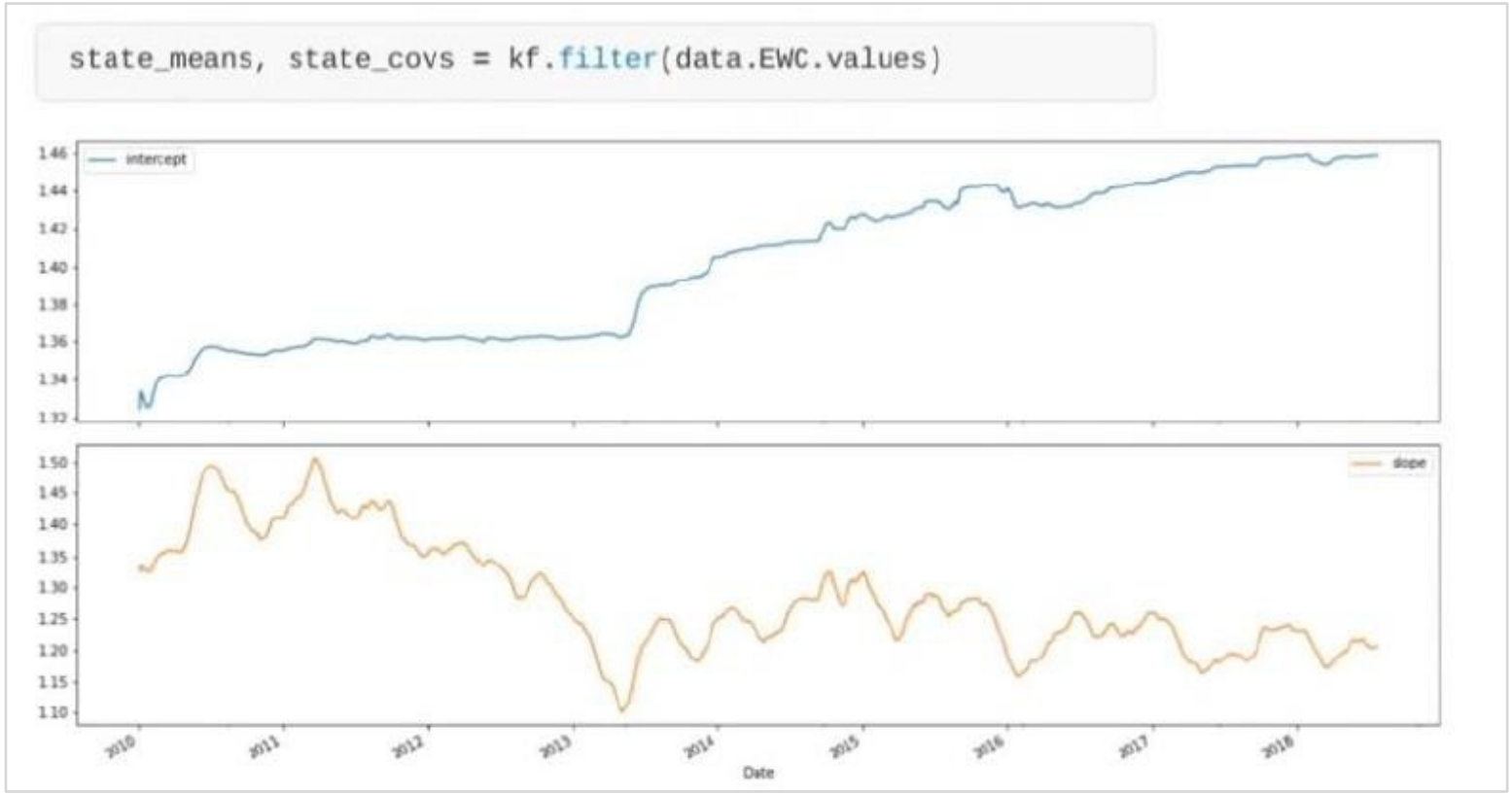
* Source: http://www.thealgoengineer.com/2014/online_linear_regression_kalman_filter/

Filtering EWC gives a mean and covariance path

Kalman filter example — mean and covariance of the predicted state on EWC

What to see

- state_means, state_covs = kf.filter(EWC)
- Top: intercept
- Bottom: slope

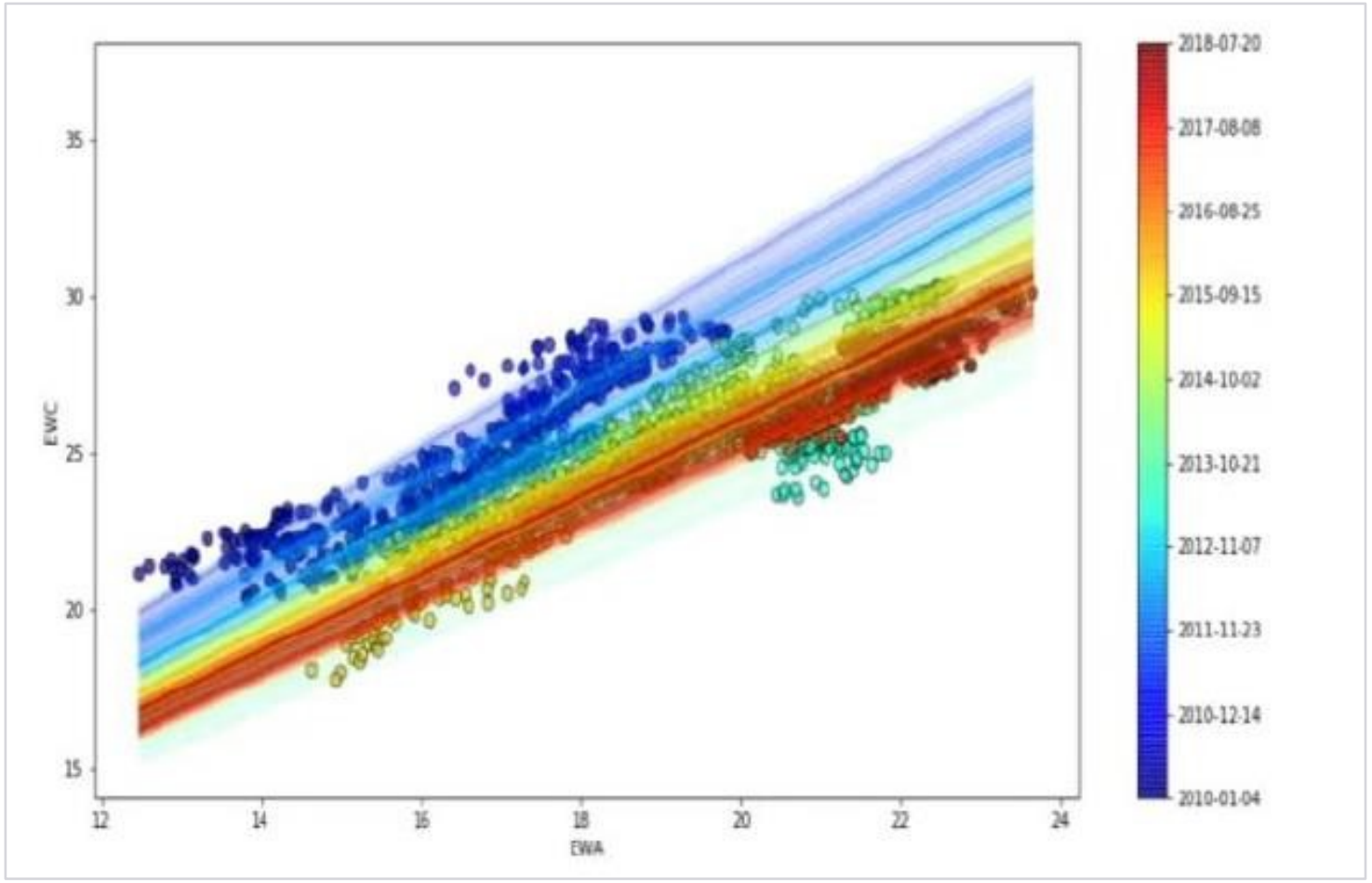


The predicted line drifts over time

Kalman filter example — predicting EWA from EWC

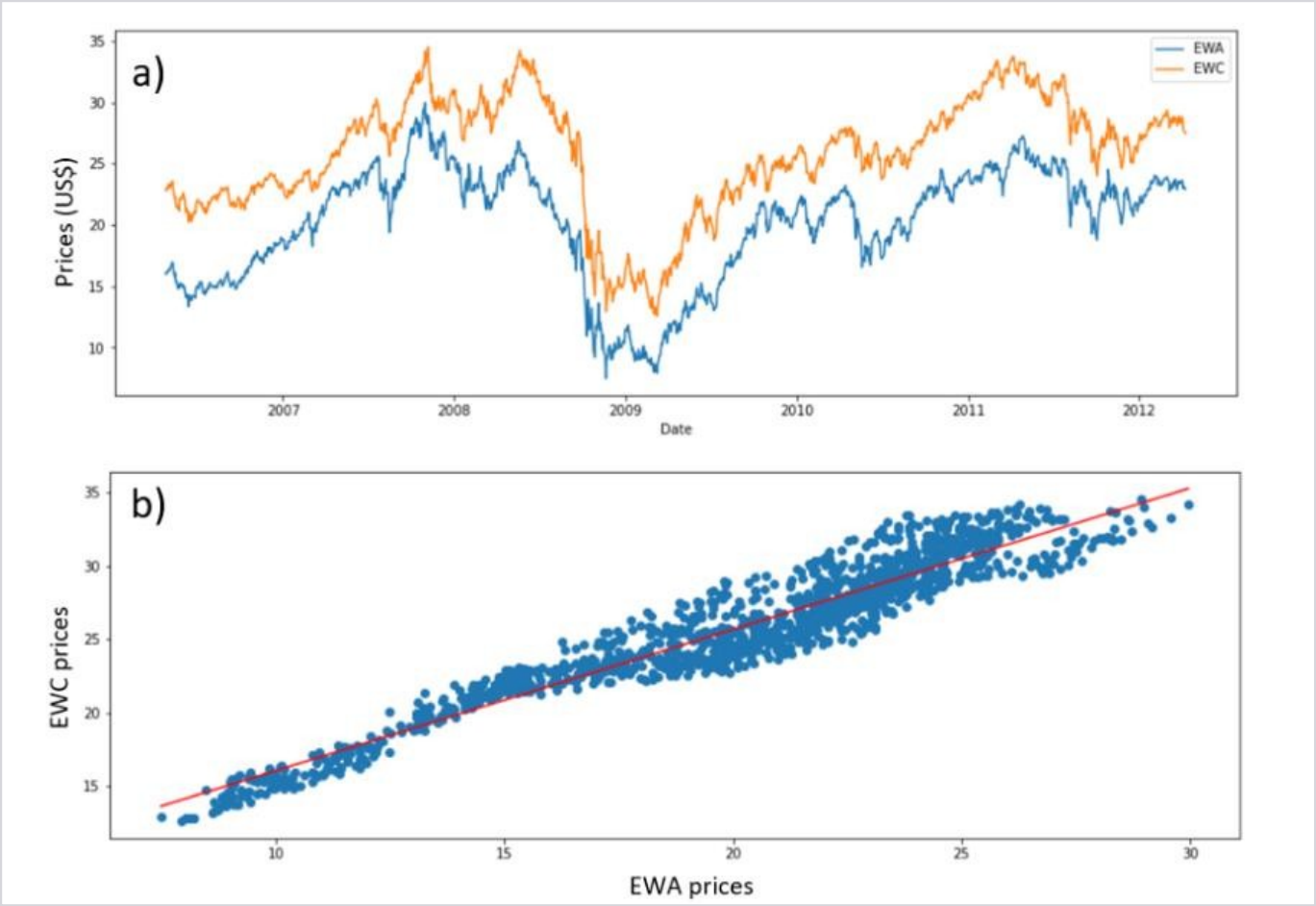
What to see

- The regression line of the correlation forecast changes over time
- Color = date



A static OLS hedge ratio links EWC to EWA

Ernest Chan's pairs trading example — EWA vs. EWC



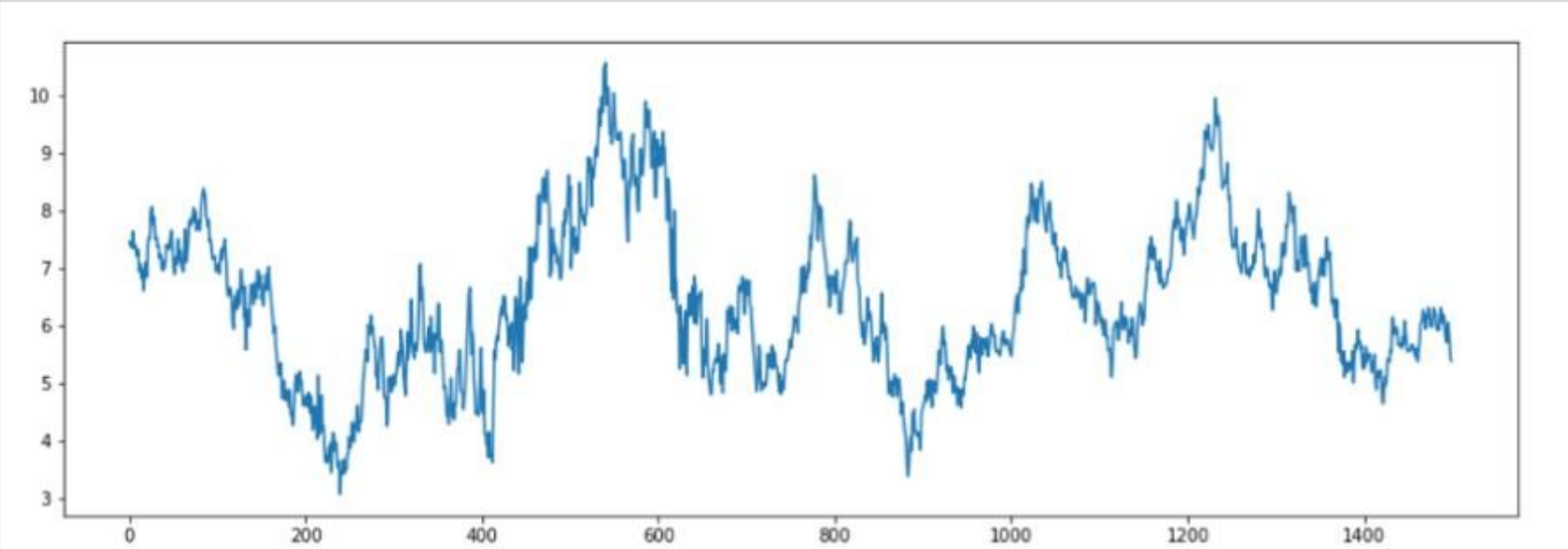
Fixed hedge ratio

$$EWC = \beta EWA + \alpha$$

$$EWC - 0.9624 EWA = 6.4113$$

The fixed-ratio spread is the trading signal

Ernest Chan's pairs trading example — the spread



$$EWC = \beta EWA + \alpha$$

$$EWC - 0.9624EWA = 6.4113$$

The hedge ratio becomes a random-walk state

Kalman filter pairs trading — state and observation equations

State equation

$$x_t = Ax_{t-1} + w_t, \quad A = I_2$$

$$\begin{pmatrix} \beta_t \\ \alpha_t \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \beta_{t-1} \\ \alpha_{t-1} \end{pmatrix} + w_t$$

State vector

$$x_t = \begin{pmatrix} \beta_t \\ \alpha_t \end{pmatrix}$$

Observation equation

$$z = Hx + v, \quad x = (\beta, \alpha)^T$$
$$H = (EWA, 1)$$

In words

$$EWC = EWA \cdot \beta + \alpha + v$$

Prices are observed; the hedge ratio is the state

Kalman filter pairs trading — relax the fixed hedge ratio assumption

1. State prediction

$$A = I \text{ (identity matrix)}$$

$$\hat{x}_t = x_{t-1}$$

2. Uncertainty prediction

$$\hat{P}_t = P_{t-1} + Q$$

3. Kalman gain

$$H = \begin{pmatrix} EWA \\ 1 \end{pmatrix}^T$$

$$K_t = \hat{P}_t H^T (H * \hat{P}_t * H^T + R)^{-1}$$

Note

$$z = Hx + v, \qquad x = \begin{pmatrix} \beta \\ \alpha \end{pmatrix}$$

Chan's settings fix the process and noise levels

Process noise Q (2×2) and measurement noise R (1×1), initialized as Chan suggests

$$Q = \frac{\delta}{1-\delta} I, \text{ being } \delta = 0.0001$$

$$R = 0.001$$

Small δ → hedge ratio changes slowly; R is the noise of the price observation

pykalman's KalmanFilter maps onto the model

Instantiate the KalmanFilter class from the pykalman module

```
kf = KalmanFilter(n_dim_obs=1, n_dim_state=2,
                  initial_state_mean=np.zeros(2),
                  initial_state_covariance=np.ones((2, 2)),
                  transition_matrices=np.eye(2),
                  observation_matrices=obs_mat,
                  observation_covariance=1.0,
                  transition_covariance=trans_cov)
```

$$
\begin{aligned}
x_0 &\sim \text{Gaussian}(\mu_0, \Sigma_0) \\
x_{t+1} &= A_t x_t + b_t + \epsilon_{t+1}^1 \\
y_t &= C_t x_t + d_t + \epsilon_t^2 \\
\epsilon_t^1 &\sim \text{Gaussian}(0, Q) \\
\epsilon_t^2 &\sim \text{Gaussian}(0, R)
\end{aligned}
$$

Parameter Name	Notation
initial_state_mean	μ_0
initial_state_covariance	Σ_0
transition_matrices	A
transition_offsets	b
transition_covariance	Q
observation_matrices	C
observation_offsets	d
observation_covariance	R

The class stores R, Q, state and covariance

Pairs trading — Python implementation (1/3)

```python
class MyKalmanFilter:

    def __init__(self, delta=1e-4, R=1e-3):
        # measurement noise variance
        self.R = R

        # co-variance of process noise(2 dimensions)
        self.Q = delta / (1-delta) * np.eye(2)
        # state (slope, intercept) will be (2 x n),
        # we will initialize with just one column at first
        self.x = np.zeros((2, 1))

        # state covariance
        self.P = np.zeros((2,2))
```

How to read the code

`class MyKalmanFilter`	Defines the filter's values and calculations as one bundle.
`def __init__`	Runs once when the filter is created. If delta and R are not given, the values after = are used.
`self.`	A value this filter object remembers, for use in the next time step.
`self.R = R`	Stores R, the variance of the measurement noise.
`self.Q = ...`	Process noise Q = δ/(1−δ) × identity (slide 50). np.eye(2) is the 2×2 identity matrix.
`self.x = ...`	State (slope, intercept). np.zeros((2, 1)) is a 2×1 column of zeros.
`self.P = ...`	State covariance. It starts as a 2×2 matrix of zeros.

step_forward runs the time update and gain

Pairs trading — Python implementation (2/3)

```python
def step_forward(self, y1, y2):
    # Before entering the equations, let's define H as (1, 2) matrix
    H = np.array([y2, 1])[None]
    # and define z
    z = y1

    ## TIME UPDATE ##
    # first thing is to predict new state as the previous one (2x1)
    x_hat = self.x[:, -1][..., None]

    # then, the uncertainty or covariance prediction
    P_hat = self.P + self.Q

    ## MEASUREMENT UPDATE ##
    # calc the Kalman gain
    K = P_hat.dot(H.T)/(H.dot(P_hat.dot(H.T))+self.R)
```

How to read the code

`step_forward(self, y1, y2)`

One day of calculation. y1 is the EWC price, y2 the EWA price.

`H = ...` $H = (EWA,\ 1)$

[None] makes it a 1×2 matrix.

`z = y1`

Today's measurement.

`x_hat = ...` $\hat{x}_t = x_{t-1}$

Takes the last column of self.x (the previous state) as a 2×1 column.

`P_hat = ...` $\hat{P}_t = P_{t-1} + Q$

Adds Q to the previous self.P, so the uncertainty grows.

`K = ...` $K_t = \hat{P}_t H^T (H \hat{P}_t H^T + R)^{-1}$

.dot = matrix product, .T = transpose. The denominator is 1×1, so / works.

* Source: http://www.thealgoengineer.com/2014/online_linear_regression_kalman_filter/

The measurement update corrects x and P

Pairs trading — Python implementation (3/3)

```
# state update part 1 (measurement estimation)
z_hat = H.dot(x_hat)
# state update part 2
x = x_hat + K.dot(z-z_hat)

# uncertainty update
self.P = (np.eye(2)-K.dot(H)).dot(P_hat)

# append the new state to the vector
self.x = np.concatenate([self.x, x], axis=1)

return x, self.P, K, z_hat
```

Why self.P?

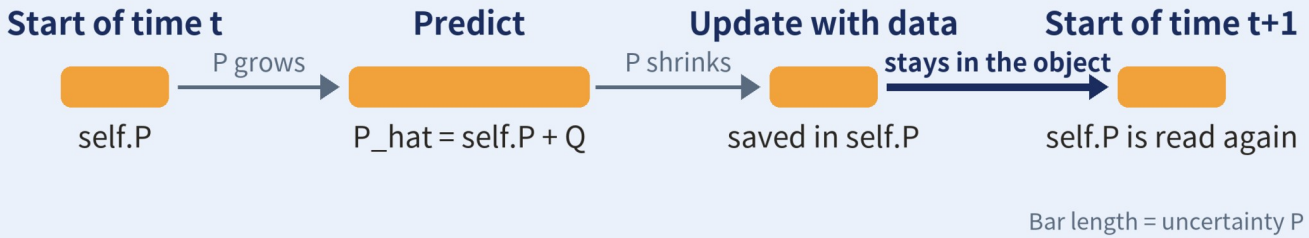
① P says how far the estimate can be trusted (its uncertainty).

② The Kalman filter does not stop after one calculation. The P found now is used again at the next time step.

③ So it is saved inside the filter object, where it survives after the function ends. That storage slot is named self.P.

④ K and z_hat, without self., are used in this calculation only. They stay out of the object and are recomputed next time.

x is also a local variable, but its value is added to self.x as a new column. self.P is overwritten with the latest value.



* Source: http://www.thealgoengineer.com/2014/online_linear_regression_kalman_filter/

step_forward returns four values

return x, self.P, K, z_hat: meaning, equation and code line for each value

Returned	Meaning	Equation	Code line
`x`	The state estimate just updated	$x_t = \hat{x}_t + K_t(z_t - H\hat{x}_t)$	`x = x_hat + K.dot(z-z_hat)`
`self.P`	The updated uncertainty of the estimate	$P_t = (I - K_t H)\hat{P}_t$	`self.P = (np.eye(2)-K.dot(H)).dot(P_hat)`
`K`	The Kalman gain	$K_t = \hat{P}_t H^T (H\hat{P}_t H^T + R)^{-1}$	`K = P_hat.dot(H.T)/(H.dot(P_hat.dot(H.T))+self.R)`
`z_hat`	The predicted measurement	$\hat{z}_t = H\hat{x}_t$	`z_hat = H.dot(x_hat)`

self.x = np.concatenate([self.x, x], axis=1): adds the new state as a column on the right of self.x. The next x_hat takes this last column.

np.eye(2) is I in the equation and .dot is matrix multiplication. The spread on the next slide uses the returned z_hat.

Looping over rows yields spread and hedge ratio

Pairs trading — prompt to run the class

[Input] Prompt · Claude Code (Code tab)

```
In the same folder, continue as follows.
1. Reuse the MyKalmanFilter class from the previous slides.
2. Download daily EWA and EWC closes for 2006–2012
   into two columns of df.
3. Create MyKalmanFilter(delta=1e-4, R=1e-3).
4. For each day, pass EWC and EWA to step_forward and
   collect the spread: EWC minus the prediction z_hat.
5. Plot the spread without the first 10 points, and plot
   the hedge ratio over time.
```

[Check] What to look for

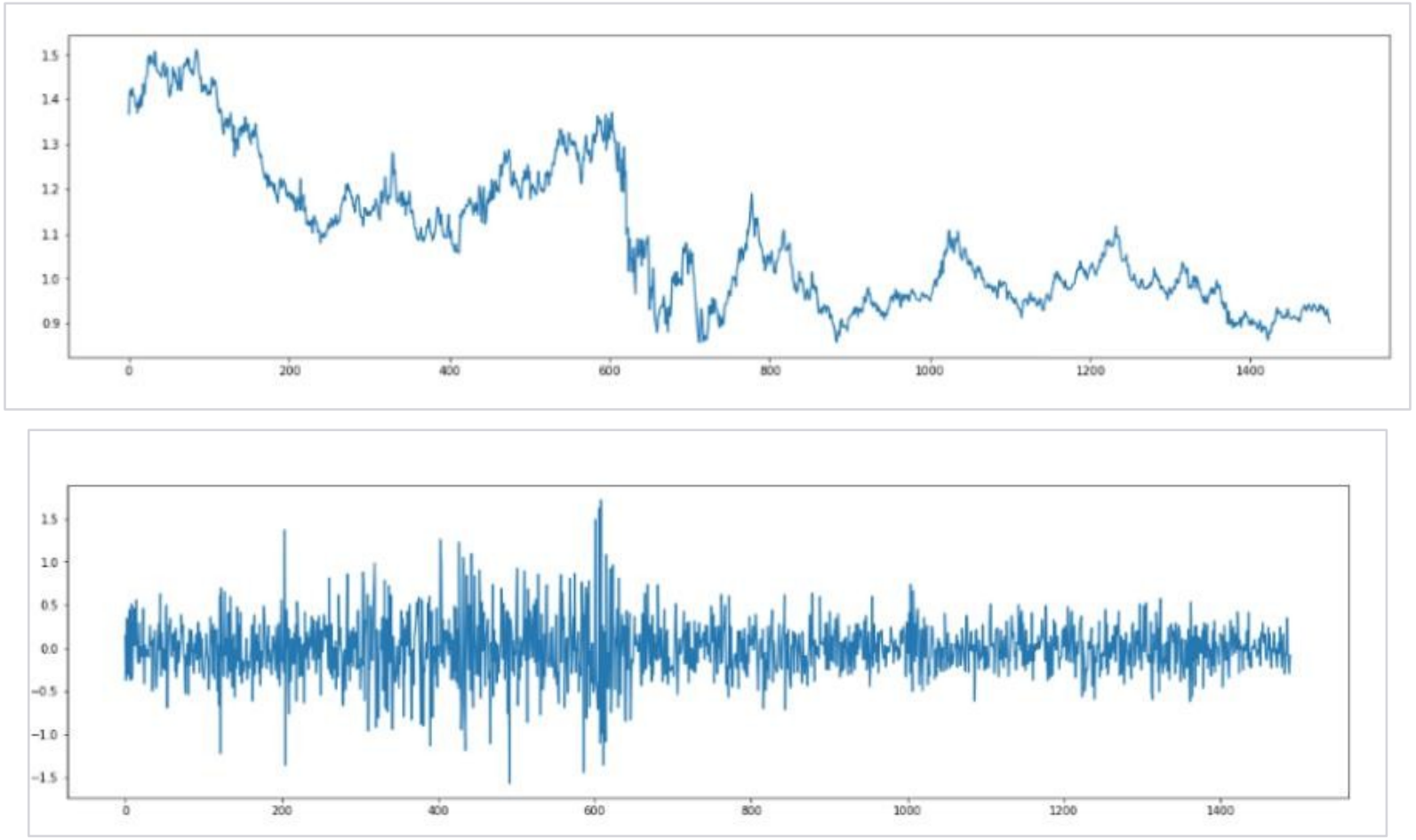
· Does the hedge ratio change over time?

· Does the spread revert to its mean faster than the fixed-ratio case (slide 47)?

* Source: http://www.thealgoengineer.com/2014/online_linear_regression_kalman_filter/

The Kalman spread reverts to its mean faster

Hedge ratio and spread over time — faster mean reversion than the fixed-ratio case



06

PART 6

Example: VIX and KOSPI

Smooth plus mean-reverting decomposition of market variables

A series = random-walk trend + AR(1) noise

Decompose an observed variable (price, credit spread, volatility) into smooth X and mean-reverting Y

$$s_t = X_t + Y_t$$

$$X_t = X_{t-1} + \varepsilon_t, \varepsilon_t \sim N(0, \sigma_\varepsilon^2)$$

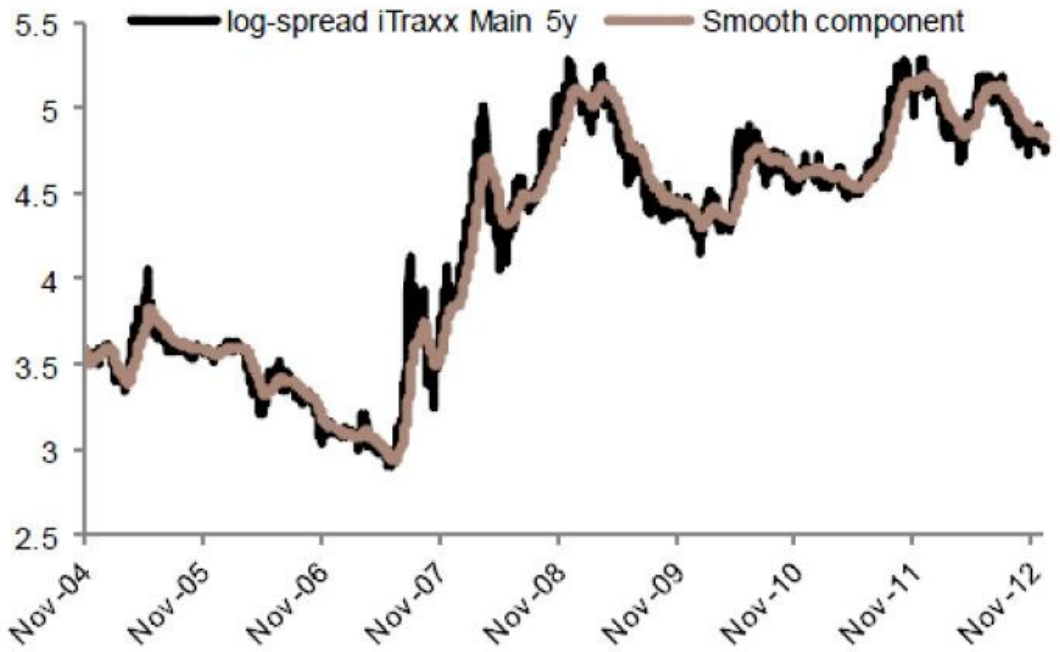
$$Y_t = \kappa Y_{t-1} + \xi_t, 0 < \kappa < 1, \xi_t \sim N(0, \sigma_\xi^2)$$

where s_t is the log value at time t,
X_t is a random walk process, and Y_t is an AR(1) process.

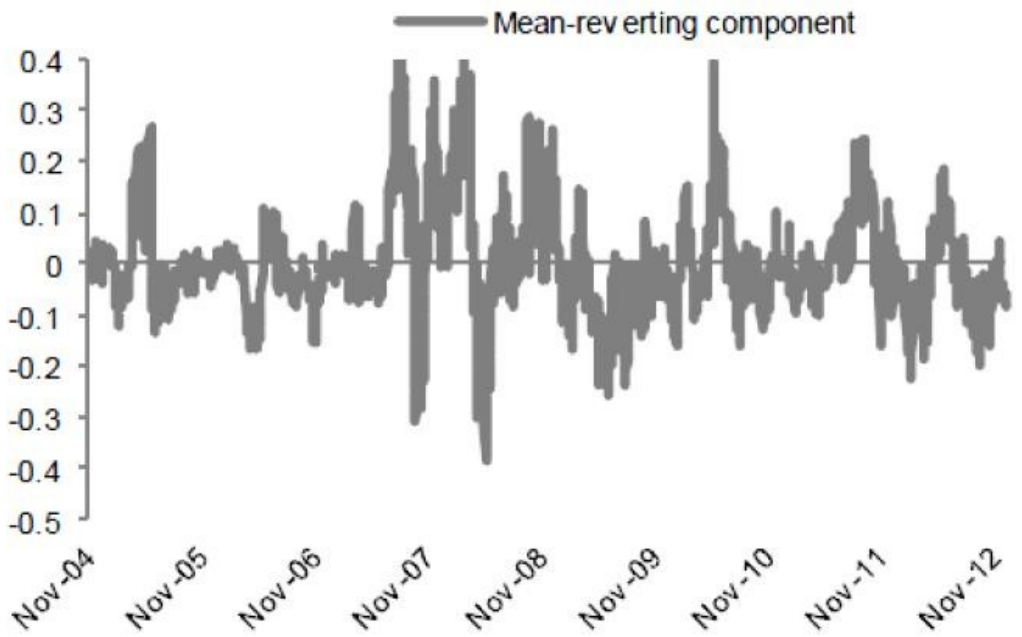
The smooth part tracks the credit spread level

Smooth and mean-reverting components (credit spread, iTraxx Main 5y)

Decomposition of log spread into a smooth component...



...and a mean-reverting component

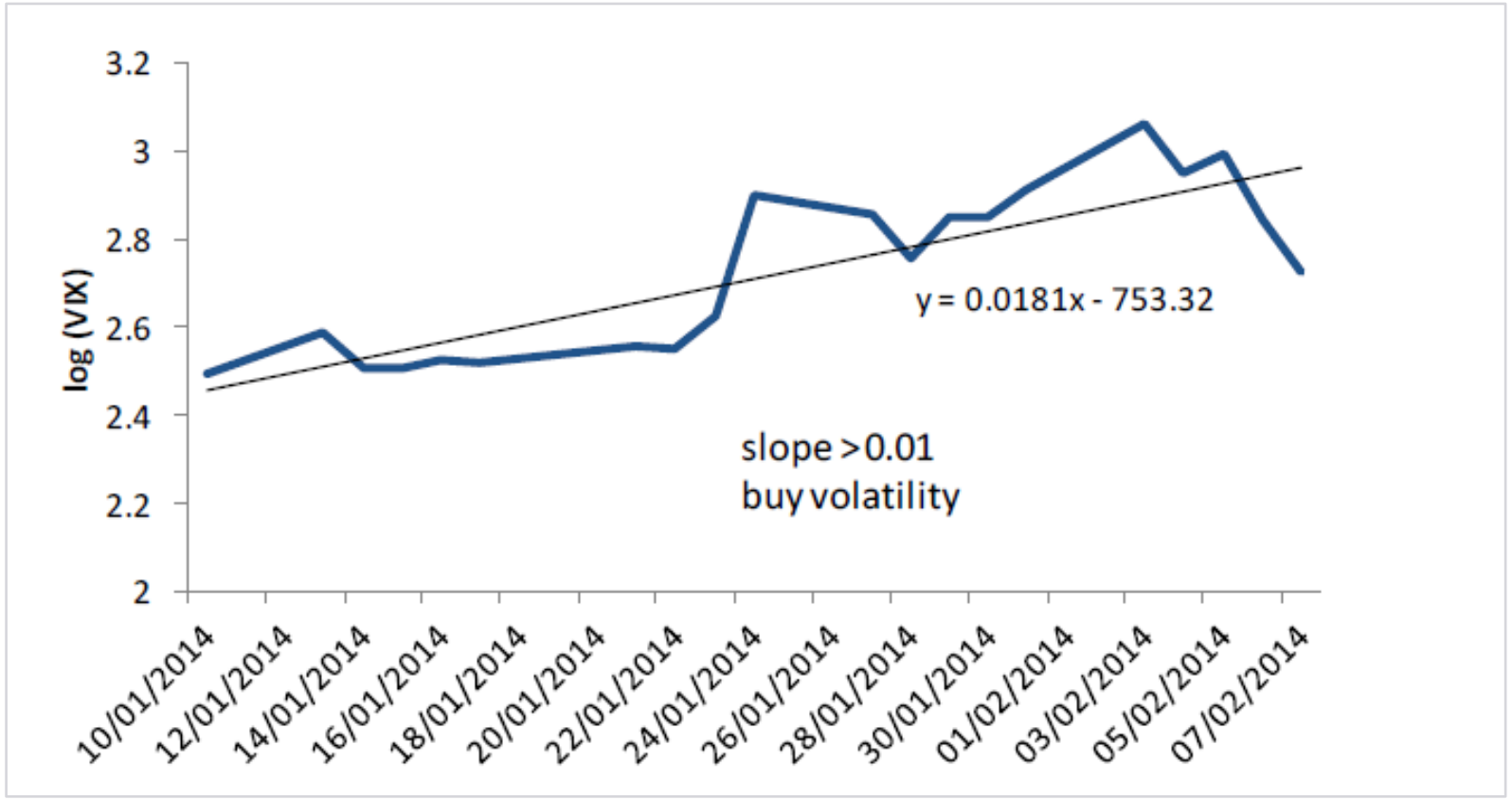


A rising VIX trend is a buy-volatility signal

Smooth component (VIX)

Reading the chart

- Log VIX smooth component with a linear trend
- Slope > 0.01 → buy volatility

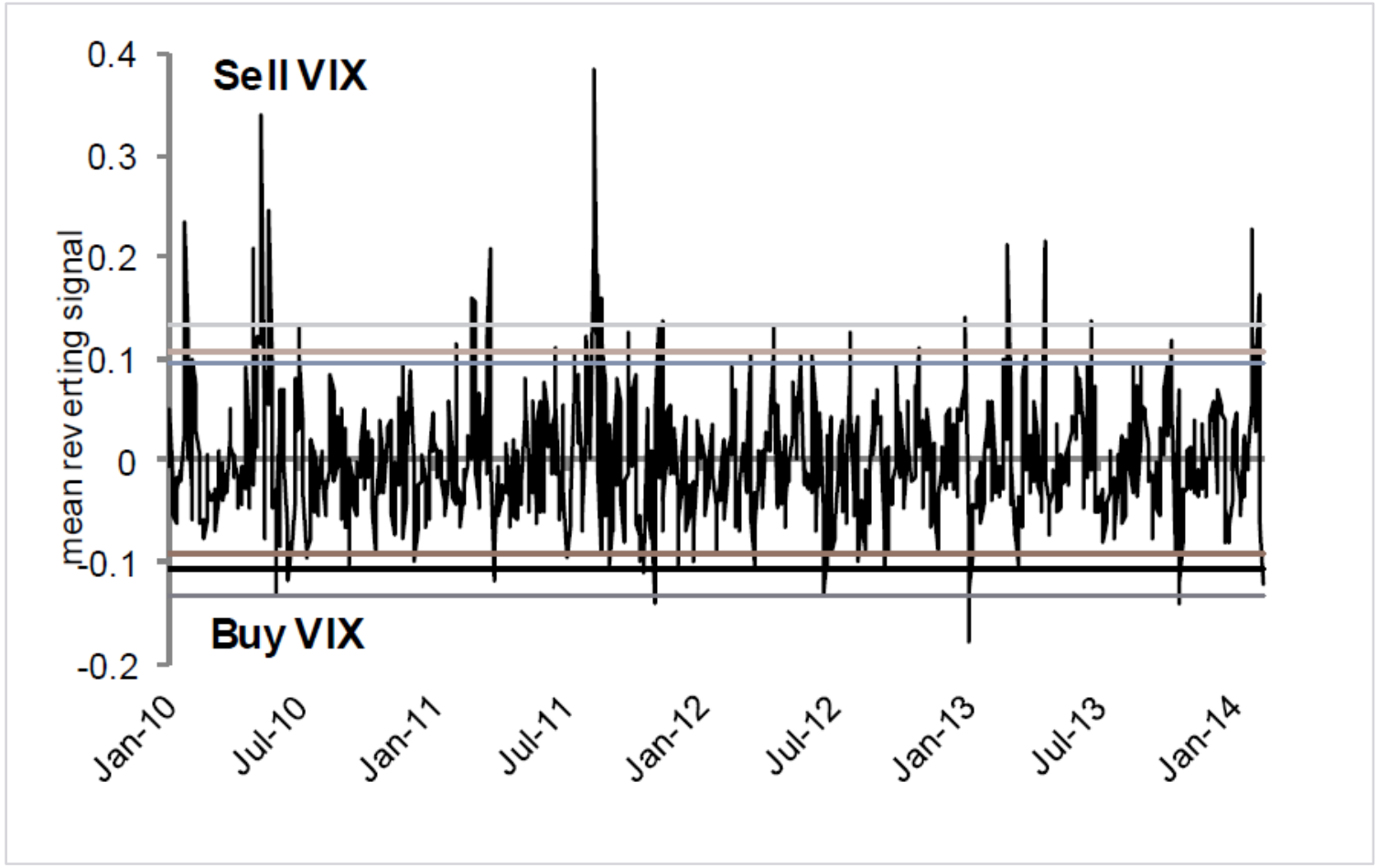


Mean-reverting extremes flag VIX trades

Mean-reverting component (VIX)

Reading the chart

- High mean-reverting signal → sell VIX
- Low signal → buy VIX

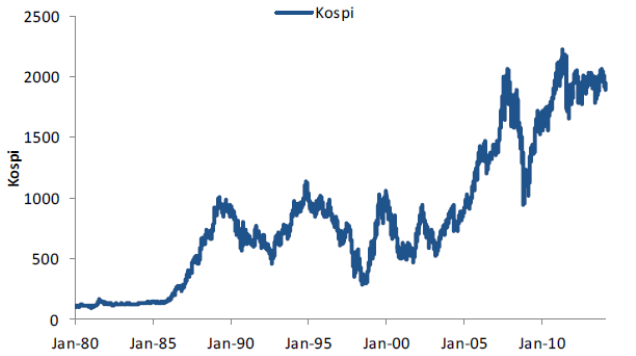


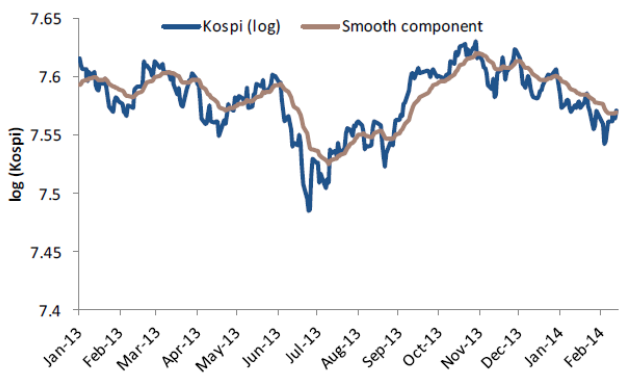
The same split applies to the KOSPI

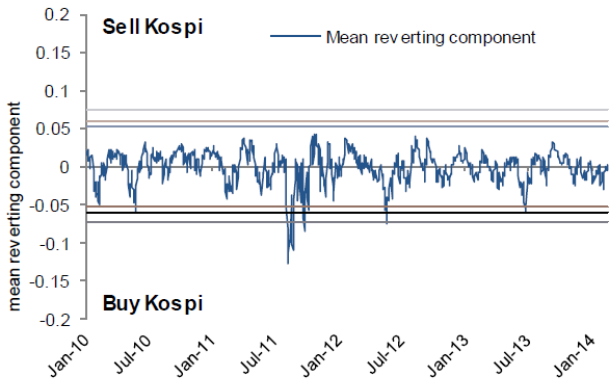
KOSPI — level, smooth component and mean-reverting component

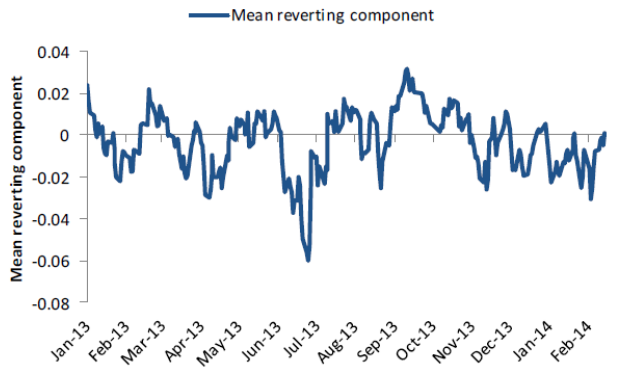
Four panels

- KOSPI level
- Log KOSPI with smooth component
- Mean-reverting component with sell / buy bands
- Mean-reverting component









Kalman Filters lecture is the main reference

References

Kalman Filters

- By Evgenia “Jenny” Nitishinskaya, Dr. Aidan O'Mahony, and Delaney Granizo-Mackenzie. Algorithms by David Edwards.